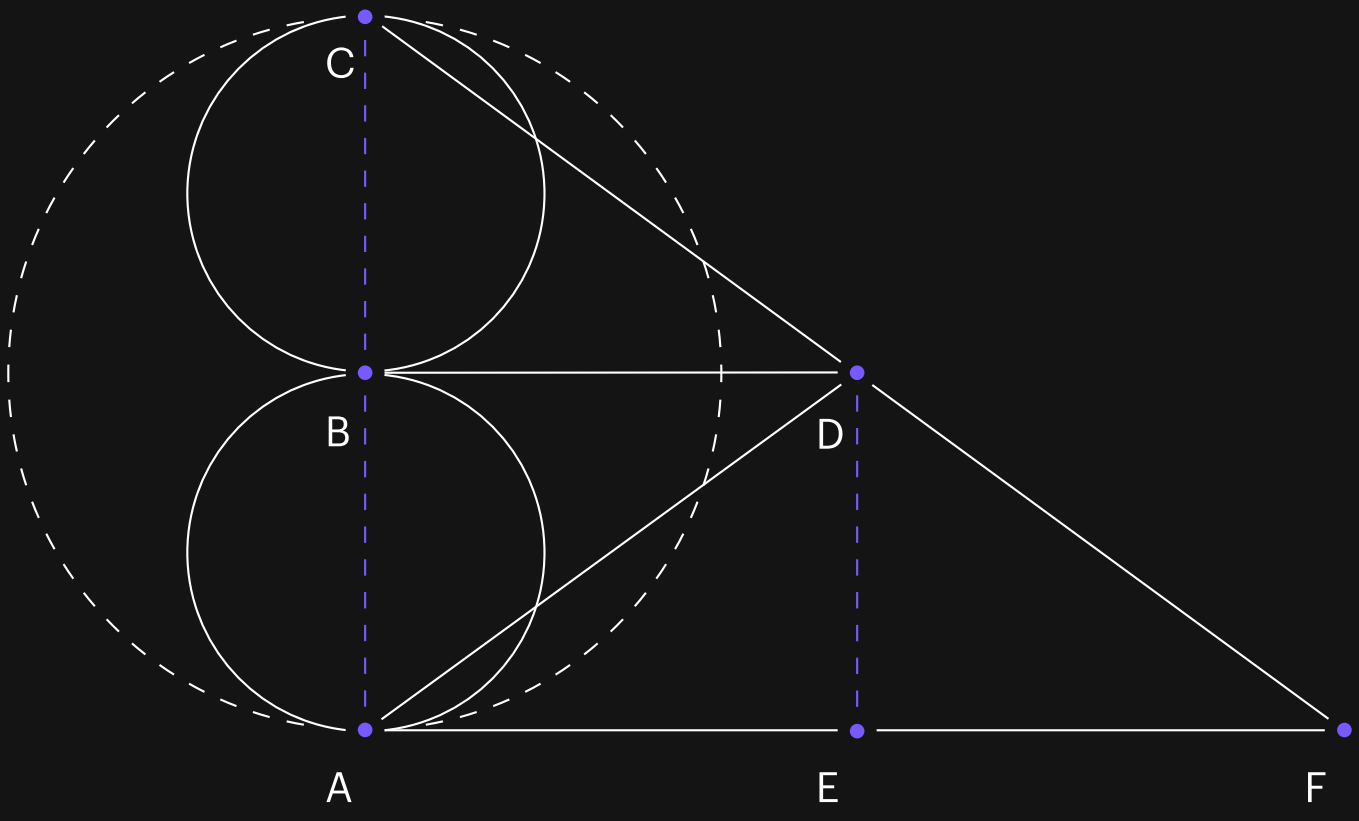
Машинное обучение

Лонгрид

Неделя 7

Сложность, переобучение и ансамбли



Содержание

1. Обобщающая способность, переобучение и сложность ML-моделей 5
1.1. Статистические основы 6
1.2. Оценка параметров в статистике 8
1.3. Оценка параметров в ML, bias-variance-noise 9
1.4. Методы борьбы с переобучением 24
1.5. Выводы по блоку 25
1.6. Переобучение деревьев решений 26
1.7. Pre-pruning-методы 26
1.8. Post-pruning-методы 27
1.9. Выводы первой части 29
2. Ансамбли 29
2.1. Почему одиночной модели недостаточно 29
2.2. Ансамбли как лекарство 30
2.3. Как обучается и работает ансамбль 32
2.4. Математика за разнообразием 34
3. Bagging и его семейство 38
3.1. Bagging 38
3.2. Random Forest 46
4. Стекинг и блендинг 47
4.1. Как работает стекинг 50
4.2. Как работает блендинг 57
4.3. Стекинг против блендинга 60
5. Если кратко 61

Сложность, переобучение и ансамбли

Контекст

В ML модели обучают на имеющихся данных, но рассчитывают, что они будут хорошо работать и на новых данных. Но здесь есть две крайности — это переобучение, когда модель запоминает шум обучающей выборки, и недообучение, когда модель слишком проста и не улавливает закономерности. Чтобы держать баланс, нужно понимать, из чего складывается ошибка модели и как на неё влияет сложность алгоритма. Чтобы проанализировать ошибку, используют идею bias-variance.

Справиться с переобучением помогают ансамбли — они объединяют множество моделей так, чтобы их сильные стороны компенсировали слабости, а случайные ошибки усреднялись.

Применение

Ансамбли — один из самых практичных ML-инструментов. Их применяют в скоринге, рекомендациях, медицинской диагностике, прогнозировании спроса и поиске аномалий. На табличных данных ансамбли деревьев и стекинг стабильно дают одни из лучших результатов и постоянно встречаются в продакшене и на соревнованиях (Kaggle). А понимание bias-variance нужно везде, где подбирают сложность модели: от регуляризации и подрезки деревьев до выбора архитектуры сети.

Краткий план

- → Обобщающая способность, переобучение и сложность ML-моделей
- → Ансамбли
- → Bagging и его семейство
- → Стекинг и блендинг
- → Если кратко

Обозначения

- x — объект (вектор признаков).
- y — целевая переменная.
- $f(x)$ — истинная зависимость.
- $\hat{f}(x)$ — обученная модель (случайная величина: зависит от того, какая обучающая выборка попалась).
- ε — неустранимый шум.

Для ансамблей:

- $b_1(x), \dots, b_K(x)$ — базовые модели («базисы»);
- F — метаалгоритм (агрегатор), собирающий их ответы в один; если он сам обучается, как в стекинге, его называют метамоделью;
- $a(x) = F\big(b_1(x), \dots, b_K(x)\big)$ — ансамбль;
- K — число базисов,
- N — число наблюдений,
- $T(x)$ — дерево.

Индексы:

- k — номер базиса (если нужен второй, берём l),
- i — номер наблюдения,
- p — число признаков,
- m — сколько признаков доступно модели при разбиении узла.

1. ОБОБЩАЮЩАЯ СПОСОБНОСТЬ, ПЕРЕОБУЧЕНИЕ И СЛОЖНОСТЬ ML-МОДЕЛЕЙ

Разберём сложности и переобучение разных ML-алгоритмов, а затем проанализируем переобучение деревьев.

ПРИМЕЧАНИЕ

Две разных «сложности». Разбирая на прошлой неделе деревья, мы считали алгоритмическую сложность — во сколько операций обходится обучение и предсказание. Эта сложность — про время и память. То есть дождёшься ли ты конца обучения.

На этой неделе «сложность» значит другое. А именно, гибкость или ёмкость модели — насколько разнообразные зависимости она способна описать. Меряют такую сложность не операциями, а числом параметров и степеней свободы: глубина дерева, степень полинома, число нейронов. Скорость обучения и гибкость не зависят друг от друга: линейная регрессия обучается быстро и предсказывает почти мгновенно, но по гибкости бедна — прямая и есть прямая. Дерево с глубиной 20 тоже обучается недолго. Но зато настолько гибкое, что запоминает шум.

Далее в лонгриде, когда мы говорим о сложности модели и о компромиссе смещения — разбросе (bias-variance), речь всегда про гибкость. Если сложность будет означать что-то другое — стоимость вычислений или трудоёмкость реализации, это будет сказано явно.

Работа с алгоритмами машинного обучения включает два шага.

1. Выбор модели — сравнение разных моделей с разными гиперпараметрами. Обычно для этого используют валидационную выборку — это помогает выбрать модели с наименьшей валидационной ошибкой.
2. Оценка модели на тестовых данных — оценка тестовой ошибки (test error). При этом её не используют для подбора лучшей модели — здесь проверка на тестовых данных служит лишь для оценки качества модели.

Остановимся на первом пункте: выборе модели и оптимального алгоритма. Наша задача — максимизация бизнес-метрики и минимизация ошибки модели. Но если наша задача — минимизация ошибки, то почему мы не можем просто построить модель, которая будет идеально проходить через «точки» обучающей выборки, как на рисунке 1?

У такого метода даже есть своё название — интерполяция.

ОПРЕДЕЛЕНИЕ

Интерполяция (в математике) — это техника восстановления функции по текущему дискретному набору её известных значений.

При построении модели метод интерполяции действительно будет минимизировать ошибку. И, по сути, обнулит ошибку на обучающей выборке (training error).

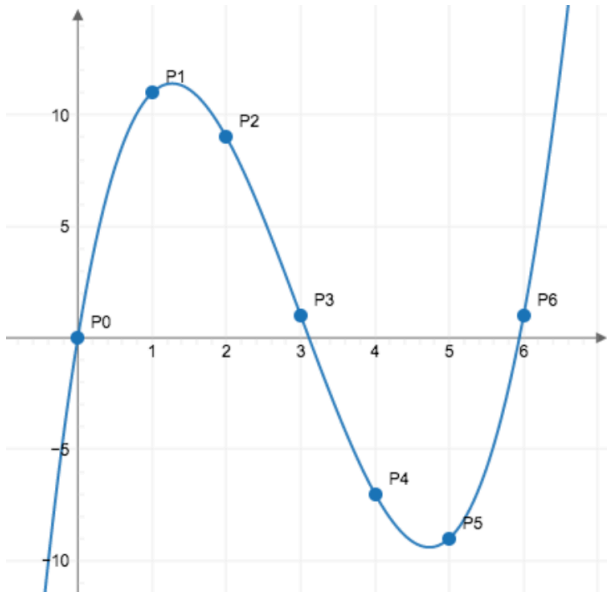


Рис. 1. Пример интерполяции по существующим числовым данным. Источник

Однако наша задача — минимизация не обучающей, а тестовой ошибки. Мы строим и подбираем модель машинного обучения для того, чтобы она хорошо предсказывала результаты на будущих данных. На практике же метод интерполяции обучающих объектов приводит к высоким ошибкам на тестовой выборке, что говорит о плохой обобщающей способности модели. Это происходит потому, что у любой выборки из данных по популяции есть шум, характерный только для этой выборки.

Но чрезмерно извилистая/гибкая модель «ловит» этот шум, путая его с истинными зависимостями в данных. Это называется переобучением (overfitting) модели на обучающей выборке, и это одна из главных проблем в машинном обучении.

В то же время и построение чрезмерно стабильной модели (например, линейной) может обернуться тем, что алгоритм не сможет подстроиться под реальную зависимость. Это уже приведёт к недообучению (underfitting).

Поэтому важно сохранить баланс и избегать крайностей. Наша цель — точно уловить истинную зависимость случайных величин, но при этом не уйти в дотошное подражание обучающей выборке.

Рассмотрим обобщающую способность моделей машинного обучения с математической точки зрения и объясним это явление.

1.1. Статистические основы

Дальше нам понадобится разложить ошибку модели на составные части. Зафиксируем три величины, которые легко перепутать:

- y — наблюдаемое значение;
- $f(x)$ — неизвестное истинное среднее отклика в точке x. То есть среднее по всем объектам с такими же признаками (именно его мы хотим угадать);
- $\hat{f}(x)$ — предсказание модели (то, что выдаёт обученная модель).

Связывает их равенство

$$y = f(x) + \varepsilon,$$

где ε — отклонение конкретного объекта от среднего.

Модель приближает $f(x)$, а сравнивать её предсказания приходится с y — отсюда и возникает неустранимая часть ошибки.

Различать эти величины будем по всему тексту. Разберём их на примере ниже.

ПРИМЕЧАНИЕ

Возьмём одну точку x — например, конкретного человека с ростом 180 см — и спросим, чему равен его вес.

1. y — вес конкретного человека из наших данных. Случайная величина: у другого человека того же роста вес будет другим.
2. $f(x)$ — средний вес всех людей ростом 180 см. Это не случайная величина, а фиксированное (но неизвестное нам) число. Именно его мы и хотим угадать.
3. $\hat{f}(x)$ — тоже случайная величина, но случайность здесь другой природы: она возникает из-за того, что модель обучена на конкретной выборке, а могла быть обучена на другой.

Модель приближает $f(x)$, а сравнивать её предсказания приходится с y — отсюда и возникает неустранимая часть ошибки.

Теперь вспомним из курса статистики три понятия, которыми будем пользоваться.

ОПРЕДЕЛЕНИЕ

Математическое ожидание (expectation) — среднее значение случайной величины, где роль весов играют вероятности.

Для дискретной величины это взвешенная сумма, а для непрерывной — интеграл:

$$\mathbb{E}[X] = \sum_k x_k \, p_k \qquad \text{или} \qquad \mathbb{E}[X] = \int_{-\infty}^{\infty} x \, p(x) \, dx.$$

Здесь:

- p_k — вероятность значения x_k,
- $p(x)$ — плотность распределения.

Обычное среднее арифметическое — частный случай, когда все веса одинаковы и равны $1/n$.

ПРИМЕЧАНИЕ

Оператор $\mathbb{E}[\cdot]$ считает математическое ожидание по распределению случайной величины. А когда важно подчеркнуть, какая именно случайная величина определяет распределение, то пишут $\mathbb{E}_X[\,\cdot\,]$. Это пригодится далее — усреднять будем как по шуму, так и по обучающим выборкам.

ОПРЕДЕЛЕНИЕ

Дисперсия (variance) — математическое ожидание квадрата отклонения случайной величины от её математического ожидания. Показывает, насколько сильно величина в среднем отклоняется от своего центра:

$$\mathrm{Var}(X) = \mathbb{E}\left[(X - \mu)^2\right], \qquad \mu = \mathbb{E}[X].$$

ОПРЕДЕЛЕНИЕ

Стандартное отклонение (standard deviation) — корень из дисперсии, $\sigma = \sqrt{\mathrm{Var}(X)}$. Стандартное отклонение удобно тем, что измеряется в тех же единицах, что и сама величина.

1.2. Оценка параметров в статистике

Предсказание модели — это тоже оценка, при которой по ограниченной выборке пытаются угадать неизвестное число. Поэтому рассмотрим, как в статистике измеряют качество оценки.

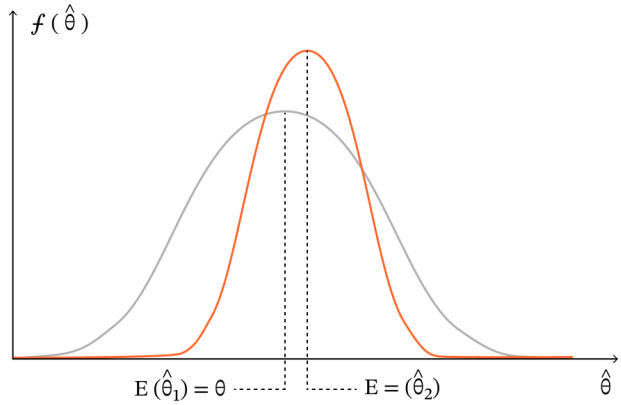


Рис. 2. Два нормальных распределения двух оценок параметра

На рисунке 2 распределения двух оценок $\hat{\theta}_1$ и $\hat{\theta}_2$ одного и того же параметра θ. Берём множество выборок и на каждой считаем оценку.

1. Первая — несмещённая, но разбросанная: её центр совпадает с истинным θ, а отдельные значения разлетаются далеко.
2. Вторая — смещённая, зато кучная (с малым разбросом).

Какая лучше — сразу не очевидно. Нужна одна общая мера, в которой смещение и разброс складываются. Чтобы найти её, возьмём квадратичный лосс $L(\theta, \hat{\theta}) = (\theta - \hat{\theta})^2$. Тогда среднеквадратичная ошибка оценки распадается ровно на два слагаемых:

$$\mathrm{MSE}(\hat{\theta}) = \mathrm{Var}(\hat{\theta}) + \mathrm{Bias}^2(\hat{\theta}), \qquad \mathrm{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

Читается это так: ошибка оценки складывается из разброса и квадрата смещения. Чем сильнее оценка разбросана и чем дальше её центр от истины, тем она хуже.

ПРИМЕЧАНИЕ

Откуда берётся $\mathrm{MSE} = \mathrm{Var} + \mathrm{Bias}^2$.

Внутри скобки прибавим и вычтем $\mathbb{E}[\hat{\theta}]$ и раскроем квадрат. Перекрёстное слагаемое обнуляется, потому что $\mathbb{E}\big[\hat{\theta} - \mathbb{E}[\hat{\theta}]\big] = 0$:

$$\begin{aligned}\mathrm{MSE}(\hat{\theta}) &= \mathbb{E}[(\hat{\theta} - \theta)^2] = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}] + \mathbb{E}[\hat{\theta}] - \theta)^2] \\ &= \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2] + (\mathbb{E}[\hat{\theta}] - \theta)^2 \\ &= \mathrm{Var}(\hat{\theta}) + \mathrm{Bias}^2(\hat{\theta}).\end{aligned}$$

Отсюда два следствия.

1. Среди несмещённых оценок лучше та, у которой разброс меньше.
2. Смещённая оценка иногда выигрывает у несмещённой, если выигрыш в разбросе перекрывает проигрыш в смещении.

Именно вторым приёмом мы будем пользоваться дальше, когда начнём подрезать деревья и добавлять регуляризацию. Чуть больше смещения в обмен на заметно меньший разброс.

ПОДСКАЗКА

Интуиция. Представь, что оценка $\hat{\theta}$ — это броски дротиков по мишени. Центр (яблочко) — истинный параметр θ. При этом один бросок = результат на одной обучающей выборке.

Тогда:

- смещение — то, насколько «облако» дротиков в среднем сдвинуто от центра,
- разброс — насколько далеко дротики расположены друг от друга.

Ошибка складывается ровно как в формуле выше:

$$\text{MSE} = (\text{насколько мимо центра})^2 + (\text{насколько разбросаны}).$$

Подробно эту картину разберём на рисунке 5.

Важно помнить, что в машинном обучении мы оцениваем не абстрактный параметр θ, а истинное среднее $f(x)$. И сравниваем предсказание не с ним, а с наблюдаемым y. Из-за этого к двум слагаемым добавится третье.

1.3. Оценка параметров в ML, bias-variance-noise

Проверим это на примере с задачей регрессии. Задача обучения с учителем (регрессии), состоит в том, чтобы в каждой точке X оценить среднее значение отклика Y. По аналогии с рисунком 2:

- оцениваемый параметр θ — это истинное среднее $f(x)$;
- оценка $\hat{\theta}$ — это предсказание модели $\hat{f}(x)$.

При этом предсказание зависит от обучающей выборки. А распределение оценки показывает, насколько сильно предсказание гуляет от выборки к выборке.

Возникают два вопроса.

1. Что именно мы оцениваем в каждой точке — наблюдаемое значение или что-то другое?
2. Если мы сможем идеально угадать эту величину, обнулит ли это ошибку модели?

СПОЙЛЕР Не обнулит. Ответ — в теме по bias-variance-noise-разложению.

Истинная зависимость

Возьмём задачу регрессии. Пусть:

- $X \in \mathbb{R}^p$ и $Y \in \mathbb{R}$;
- между X и Y существует совместное распределение (joint distribution) — $Pr(X, Y)$.

Между целевой переменной и признаками предположим зависимость вида:

$$Y = f(X) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2).$$

Здесь:

- $f(X)$ — детерминированная часть (любая функция),
- ε — случайный шум.

Условие на шум:

$$\mathbb{E}[\varepsilon \mid X] = 0.$$

В среднем шум равен нулю в каждой точке x, а не только «в целом по выборке» — это важное условие. Если бы шум систематически уходил вверх при больших x и вниз при малых, это была бы закономерность. Тогда её следовало бы отнести к $f(X)$, а не к шуму. Дополнительно обычно считают шум нормальным с постоянной дисперсией: $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$.

Таким образом, $f(X)$ — это функция нашей истинной регрессии. Наша задача — оценить значение этой функции в каждой точке x_i.

ВАЖНО

Из-за случайной величины ε в выражении невозможно идеально предсказать будущие данные. Даже лучшая оценка, равная истинной зависимости $f(X)$, будет ошибаться на величину ошибки ε. Дальше разберём, как эта неизбежная ошибка влияет на общую ошибку модели.

ПРИМЕЧАНИЕ

Два источника случайности. Далее в лонгриде ты встретишь две случайные величины — случайность отклика и случайность обучающей выборки. Они возникают по разным причинам и путать их нельзя.

	Откуда берётся	На что влияет	Как убрать
Случайность отклика ε	Даже при одном и том же x новый объект даёт другое y: рост не определяет вес однозначно	Неустранимый шум σ_ε^2	Никак — это свойство данных, а не модели
Случайность обучающей выборки	Использовали один набор данных, а могли взять другой — и модель обучилась бы иначе	Разброс предсказаний $\mathrm{Var}[\hat{f}(x)]$	Упростить модель, добавить данных, усреднить множество моделей

Как проверить себя: если мысленно зафиксировать обучающую выборку, модель станет детерминированной и разброс исчезнет. А шум останется. У большинства моделей, если собрать бесконечно много данных — разброс уйдёт к нулю, но шум снова останется на месте.

> Смещение — третье слагаемое, которое не содержит случайности. Это систематический промах модели, который остаётся, даже если усреднить её по всем возможным обучающим выборкам.

При условии квадратичного лосса $L(Y, f(X)) = (Y - f(X))^2$ истинная регрессия минимизирует Expected Prediction Error:

$$\text{EPE}(f) = \mathbb{E}\big[(Y - f(X))^2\big].$$

И тогда в каждой точке x_i истинная зависимость равна условному среднему отклика:

$$f(x_i) = \mathbb{E}[Y \mid X = x_i].$$

$f(x_i)$ — это не наблюдаемое значение y_i. Наблюдение равно $y_i = f(x_i) + \varepsilon_i$, то есть складываем условное среднее и шум конкретного объекта. Если бы в одной и той же точке x_i можно было получить много наблюдений, они бы разбросались вокруг $f(x_i)$. Именно центр этого разброса мы и хотим оценить.

Это значит, что мы стремимся к тому, чтобы наиболее точно оценить математическое ожидание $\mathbb{E}[Y \mid X = x_i]$ в каждой точке. На рисунке 3 — пример с одним категориальным признаком точек, которые оцениваем.



Рис. 3. Истинные значения зависимости $f(X)$, которые мы хотим оценить. Случайность исходит из величины ε, распределённой нормально

Зная распределения из рисунка 3, можно сразу построить идеальную модель, минимизирующую ошибку. Здесь методы машинного обучения стоит использовать в случае,

- когда неизвестно, как выглядят теоретические распределения;
- нужно оценить их свойства.

Оценка истинной зависимости. Bias-variance-noise-разложение

Наша цель — для каждой точки $X = x_i$ подобрать оценку $\hat{f}(x_i)$ истинной регрессии $f(x_i)$. Помним, что $\hat{f}(x_i)$ — это случайная величина, которая зависит от обучающей выборки расчёта оценки. Подберём такой $\hat{f}(x_i)$, который по всем возможным обучающим выборкам даст близкие к истинному значению $f(x_i)$ оценки.

Пока всё происходит по аналогии с рисунком 4.

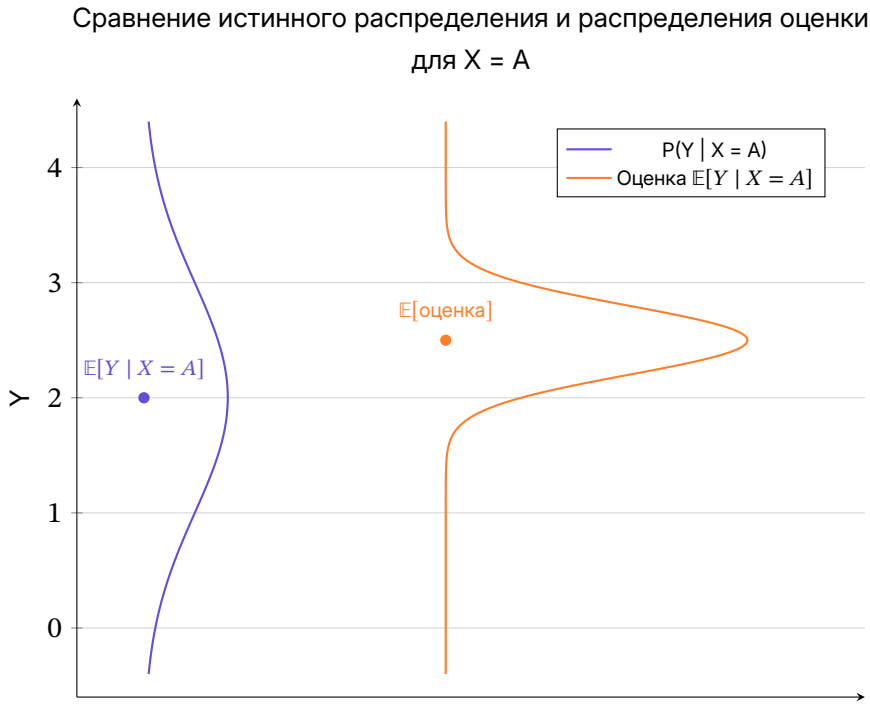


Рис. 4. Разбор условного распределения $Y|X = A$ в точке $X = A$ из предыдущего рисунка вместе с распределением оценки $\hat{E}(Y|X = A)$ истинного математического ожидания

Чтобы подобрать оптимальную оценку $\hat{f}(x_i)$, нужно понимать, из чего она состоит. В предыдущем разделе по статистическим оценкам мы вывели, что ошибка (MSE) статистических оценок состоит из суммы дисперсии оценки и её квадратичного смещения. Предположим, что мы смогли найти несмещённую оценку, которая ещё и одинакова для всех обучающих выборок. Другими словами, оценку с нулевым смещением и дисперсией. По выведенной формуле, MSE такой оценки параметра действительно равен нулю. Но ошибка предсказания отклика Y не равна нулю, и вот почему.

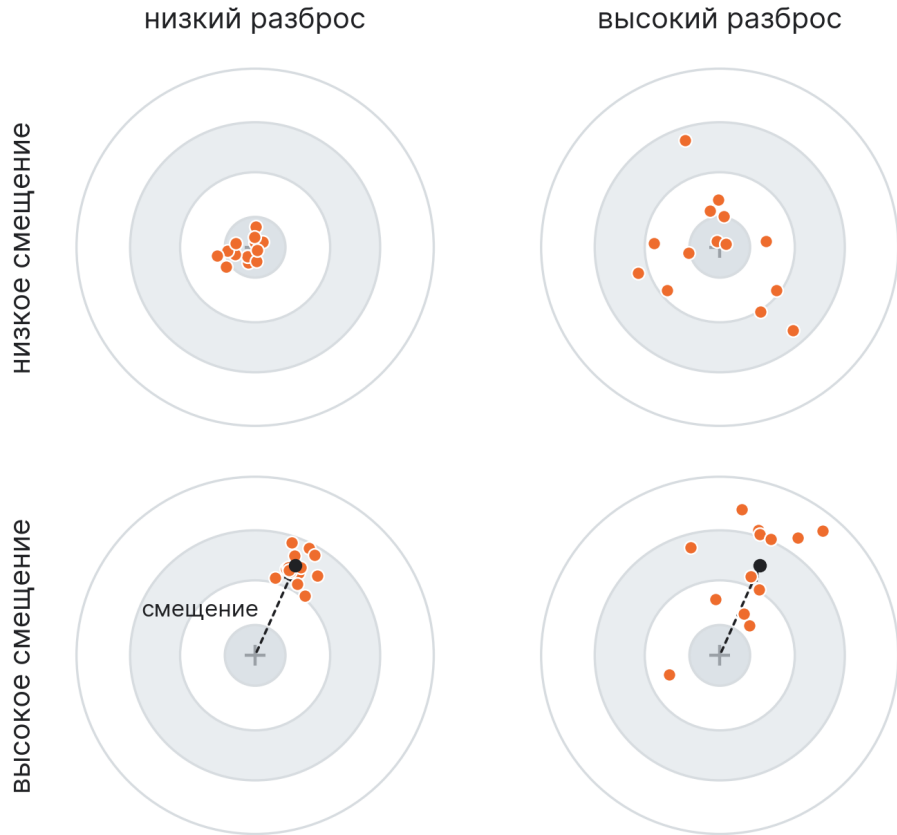
Даже если нам удастся найти оценку, равную истинному параметру (истинному математическому ожиданию), такая оценка будет давать ошибку. Дело в том, что даже истинная функция регрессии при предсказании данных выдаёт минимальный уровень ошибки ε. И если идеально оценить эту истинную регрессию, то ошибка предсказания всё равно будет ненулевой. Данные в реальном мире не детерминированные, а случайно распределённые. И даже если абсолютно точно выявить генерирующее распределение, то неопределённость в данных всё равно остаётся.

Пусть мы знаем идеальную зависимость «средний вес по росту». Даже имея эту информацию, мы не угадаем вес конкретного человека: люди отклоняются от среднего по множеству причин, которых нет в признаке «рост». Эти индивидуальные отклонения и есть ε. Никакая модель их не уберёт, потому что отклонения касаются данных, а не самой модели.

Таким образом, классическое тождество MSE $=$ Var $+$Bias^2 применимо в математической статистике только при оценке фиксированного параметра — например, среднего арифметического. Тогда как в машинном обучении мы предсказываем случайный отклик $Y = f(X) + \varepsilon$, который по своей сути имеет случайную ε-компоненту.

Это явление изображено на рисунке 4. Синее распределение — это распределение $Y|X = A$ со своим математическим ожиданием $f(X = A)$, которое оцениваем с помощью $\hat{f}(X = A)$, случайной величиной и оранжевым распределением. Распределение этой оценки находится с помощью обучения алгоритма на бесконечном количестве выборок.

Здесь оценка смещённая, так как её математическое ожидание не равно истинному параметру $\mathbb{E}[Y \mid X = A]$. В то же время разброс оценки кажется не таким большим. Но даже если бы эта оценка была детерминированной и всегда была равна $\mathbb{E}[Y \mid X = A]$, ошибка предсказания была бы ненулевая, так как синее распределение $Y|X = A$ случайно и зависит от ошибки ε (вывод из предположения о виде зависимости $Y = f(X) + \varepsilon$).



Центр мишени — истинное значение $f(x)$; оранжевая точка — одна обученная модель; чёрная — среднее предсказание. Смещение — насколько «облако» сдвинуто от центра; разброс — насколько точки разлетелись между собой.

Рис. 5. Четыре сочетания смещения и разброса. Центр мишени — истинное значение $f(x)$, каждая оранжевая точка — модель, обученная на своей выборке

Как читать «мишень»:

- каждая мишень — это одна модель;
- оранжевые точки — предсказания модели, полученные на разных обучающих выборках;
- серый крест в центре — истинное значение $f(x_0)$, которое мы хотим угадать;
- чёрная точка — среднее предсказание модели по всем выборкам;
- пунктир — смещение. В верхнем ряду смещения нет, поэтому среднее совпадает с центром и отдельной точкой не отмечено.

По столбцам меняется разброс — низкий или высокий. По строкам смещение — низкое или высокое.

Мишень	Смещение	Разброс	Комментарий
Слева вверху	Низкое	Низкий	Кучно и точно в центре, идеал
Справа вверху	Низкое	Высокий	В среднем в центре, но точки размазаны — так выглядит переобучение
Слева внизу	Высокое	Низкий	Кучно, но систематически мимо — так выглядит недообучение
Справа внизу	Высокое	Высокий	И мимо, и вразброс, худший случай

ПРИМЕЧАНИЕ

Смещение — то, насколько облако точек в среднем сдвинуто от центра. Разброс — насколько точки раскиданы между собой. Здесь хорошая модель должна быть «кучной» и «по центру» одновременно.

Посчитаем, чему равна средняя ошибка модели $\hat{f}(x_0)$ в конкретной тестовой точке $X = x_0$, из каких частей она складывается и чем отличается от MSE оценки обычного параметра.

INTERVIEW KIT

Наш выбор — квадратичный функционал ошибки. Средняя тестовая ошибка модели в точке x_0 раскладывается на три слагаемых:

$$\mathbb{E}\big[(y - \hat{f}(x_0))^2\big] = \sigma_\varepsilon^2 + \mathrm{Var}\big[\hat{f}(x_0)\big] + \mathrm{Bias}^2\big[\hat{f}(x_0)\big].$$

Здесь:

- σ_ε^2 — неустранимый шум;
- $\mathrm{Var}\big[\hat{f}(x_0)\big]$ — разброс предсказаний;
- $\mathrm{Bias}^2\big[\hat{f}(x_0)\big]$ — квадрат смещения.

Здесь подсчёт такой же, как MSE оценки в статистике, — через тождество $\mathbb{E}[X^2] = \mathrm{Var}(X) + (\mathbb{E}[X])^2$. Разница в том, что теперь случайны обе величины: и наблюдение y (из-за шума ε), и предсказание $\hat{f}$ (из-за того, какая обучающая выборка нам попалась). А дисперсия наблюдения даёт третье слагаемое σ_ε^2, которого в статистической формуле не было.

ПРИМЕЧАНИЕ

Дополнительно: полный вывод разложения. Для краткости $f = f(x_0)$, $\hat{f} = \hat{f}(x_0)$, $y = f + \varepsilon$.

Пользуемся тремя фактами:

- $\mathbb{E}[X^2] = \mathrm{Var}(X) + (\mathbb{E}[X])^2$;
- $\mathbb{E}[y] = f$, потому что шум в среднем нулевой;
- шум тестового объекта и обученную модель считаем независимыми.

Поэтому $\mathbb{E}[y\hat{f}] = \mathbb{E}[y]\,\mathbb{E}[\hat{f}]$.

$$
\begin{aligned}
\mathbb{E}\big[(y-\hat{f})^2\big] &= \mathbb{E}\big[y^2 + \hat{f}^2 - 2y\hat{f}\big] \\
&= \mathbb{E}[y^2] - \big(\mathbb{E}[y]\big)^2 + \big(\mathbb{E}[y]\big)^2 + \mathbb{E}[\hat{f}^2] - \big(\mathbb{E}[\hat{f}]\big)^2 + \big(\mathbb{E}[\hat{f}]\big)^2 - 2\,\mathbb{E}[y]\,\mathbb{E}[\hat{f}] \\
&= \mathrm{Var}(y) + \mathrm{Var}(\hat{f}) + \big(\mathbb{E}[y]\big)^2 + \big(\mathbb{E}[\hat{f}]\big)^2 - 2\,\mathbb{E}[y]\,\mathbb{E}[\hat{f}] \\
&= \mathrm{Var}(y) + \mathrm{Var}(\hat{f}) + f^2 + \big(\mathbb{E}[\hat{f}]\big)^2 - 2f\,\mathbb{E}[\hat{f}] \\
&= \mathrm{Var}(y) + \mathrm{Var}(\hat{f}) + \big(\mathbb{E}[\hat{f}] - f\big)^2 \\
&= \sigma_\varepsilon^2 + \mathrm{Var}\big[\hat{f}(x_0)\big] + \mathrm{Bias}^2\big[\hat{f}(x_0)\big].
\end{aligned}
$$

В последней строке $\mathrm{Var}(y) = \sigma_\varepsilon^2$: f — это константа, вся случайность y сидит в шуме. А $\big(\mathbb{E}[\hat{f}] - f\big)^2$ — это в точности квадрат смещения модели.

INTERVIEW KIT

Смещённость оценки показывает, насколько хорошо мы настраиваемся на целевую зависимость. В то же время разброс показывает, насколько модель зависит от шума в обучающих выборках. Разберём эту связь.

Обучающая выборка всегда одна из многих возможных. Кроме истинной зависимости, в ней есть шум, характерный только для неё. Обычно чем гибче модель, тем точнее она подстраивается под каждую точку. А значит, и под этот шум. Но в другой выборке шум был бы другим, поэтому и модель получилась бы другой. Поэтому предсказания модели в одной и той же точке x_0 различаются от выборки к выборке.

Это различие и есть $\mathrm{Var}[\hat{f}(x_0)]$ — второе слагаемое разложения. А так как тестовая ошибка равна сумме трёх слагаемых, рост разброса напрямую поднимает ошибку на новых данных, даже когда смещение мало. Отсюда вывод: гибкость достигается ценой большего разброса, а разброс ведёт к ухудшению качества на новых данных

На рисунке 5 — оценки четырёх разных моделей, где каждая оранжевая точка показывает оценку модели на конкретной обучающей выборке. Ошибка модели зависит от смещения и разброса: чем больше оба показателя, тем ниже качество предсказаний.

ПРИМЕЧАНИЕ

Хорошая оценка $\hat{f}(x_0)$ даёт значения в среднем по всем обучающим выборкам, близкие к $f(x_0)$.

Неустранимый шум остаётся даже тогда, когда истинная зависимость известна. Предположим, мы предсказываем самим $f(x)$ — идеально, без ошибки в модели. В этом случае средняя квадратичная ошибка в точке x всё равно не будет нулевой: она равна $\mathbb{E}[\varepsilon^2 \mid X = x] = \sigma_\varepsilon^2$. Это и есть неустранимая часть (irreducible error).

ПРИМЕЧАНИЕ

Неустранимая ошибка — не сам ε, а его средний квадрат. Отдельное отклонение может оказаться и большим, и почти нулевым, но в среднем по квадрату получается σ_ε^2. Остальное — $\text{Bias}^2(\hat{f}(x_0)) + \text{Var}(\hat{f}(x_0))$ — устранимо (reducible) и зависит от того, какую модель мы выбрали. Значит, минимизировать нужно именно устранимую часть.

Кстати, Expected Prediction Error (Expected Test Error) может усредняться для всех точек $X = x_i$. Таким образом, конечная тестовая ошибка модели на всех возможных точках множества величины X:

$$\text{EPE}(\hat{f}) = \mathbb{E}\big[(Y - \hat{f}(X))^2\big].$$

Эта конечная тестовая ошибка состоит из смещений, дисперсий и неустранимых шумов в предсказаниях каждой отдельной тестовой точки $X = x_i$.

Разберём, что такое смещение, разброс и естественный шум на конкретном примере (рисунок 6). Где на рисунках:

- чёрные функции — истинные зависимости Y от X;
- синие функции — наши оценки этих зависимостей на разных обучающих выборках;
- синие точки — предсказания $\hat{y}$;
- чёрные точки — истинные возможные значения y.

Вариативность в предсказаниях получается из-за разных обучающих выборок. А вариативность в истинных значениях — это зависимость от случайной ошибки ε, предположительно распределённой нормально с математическим ожиданием 0 и стандартным отклонением σ_ε.

Здесь итоговая ошибка предсказания на тестовом объекте x зависит от смещения, разброса и естественного шума. В обоих случаях данные распределены одинаково, а потому чёрные точки слева и справа совпадают. С неизбежным шумом в данных сделать ничего не можем. Но предсказания (синие точки) сильно отличаются: хотя разброс предсказаний почти одинаков в обеих моделях, смещённость оценки $\hat{y}$ в правой модели значительно ниже. Это говорит о том, что правая модель лучше настраивается на целевую зависимость. Поэтому итоговая тестовая ошибка на объекте x значительно ниже в правом случае, а значит, правая модель качественнее.

ПРИМЕЧАНИЕ

Зафиксируем, чем из разложения можно управлять.

- Шум — находится в самих данных. Он останется, даже если угадать истинную зависимость абсолютно точно. Это нижняя планка ошибки, и выбором модели её не сдвинуть.
- Смещение — систематическая ошибка модели. В среднем по всем возможным обучающим выборкам она промахивается мимо истинной зависимости, так как ей не хватает гибкости повторить форму зависимости.
- Разброс — чувствительность к обучающей выборке. Обучи ту же модель на другой выборке того же размера, и предсказания заметно изменятся.

Шум от нас не зависит, а смещение и разброс зависят — и оба сразу настраиваются сложностью модели. Поэтому выбор сложности почти всегда компромисс между двумя этими ошибками.

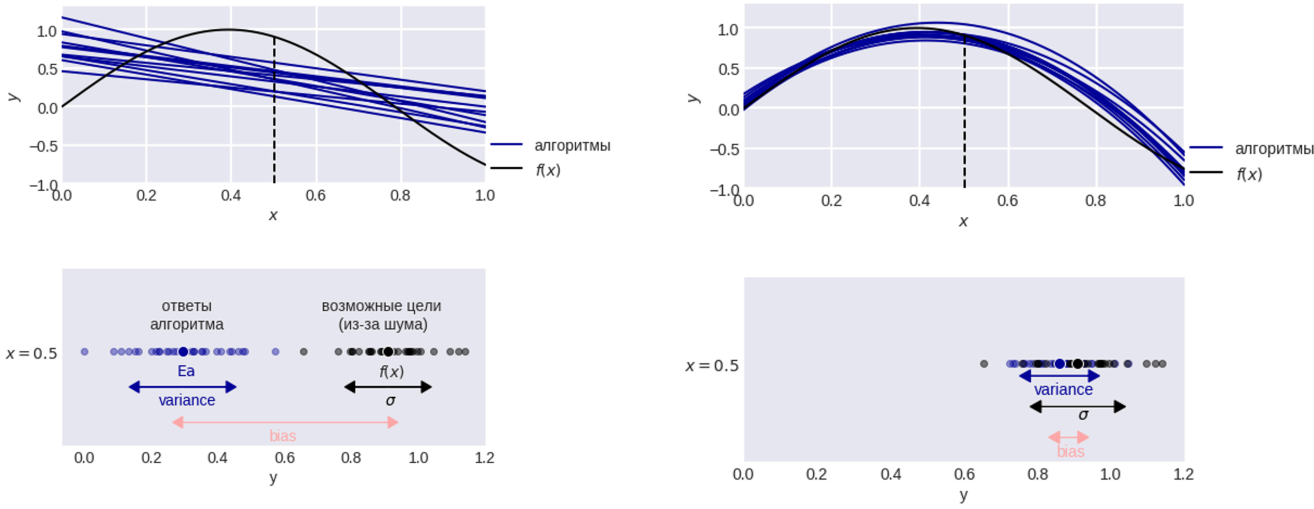


Рис. 6. Разложение тестовой ошибки на тестовом объекте x двух разных моделей. Источник

Bias-variance tradeoff

Разберём компромисс между смещением и разбросом. И посмотрим, как именно сложность модели двигает смещение и разброс, почему из-за этого кривая тестовой ошибки получается U-образной и как по ней выбирать оптимальную модель.

Выбор оптимального алгоритма зависит от его качества на данных, которых модель не видела при обучении. На практике сложность подбирают по валидационной выборке или по кросс-валидации. А тестовую выборку используют один раз в самом конце, чтобы получить честную итоговую оценку. Всё потому, что, если настраивать гиперпараметры глядя на тест, он перестаёт быть независимым и начинает завышать качество.

Наша цель — подобрать метод, минимизирующий ошибку на новых данных. Мы доказали, что тестовая ошибка состоит из трёх факторов: смещения, дисперсии оценки (устранимой ошибки) и присущего данным естественного шума (неустранимой ошибки). На естественный шум повлиять не получится, а вот подбор метода, минимизирующего смещение и дисперсию оценки, — это наша задача.

INTERVIEW KIT

Разные типы моделей различаются по сложностям — количеству параметров или степеней свободы. Регулируя сложность, подберём алгоритм, который минимизирует связку «смещение + разброс».

Проблема в том, что две части этой ошибки ведут себя противоположно. Разброс с ростом гибкости растёт — это мы уже выяснили. А смещение, наоборот, обычно падает: чем больше у модели параметров, тем в среднем она меньше ошибается. Значит, уменьшая одно, мы чаще всего увеличиваем другое.

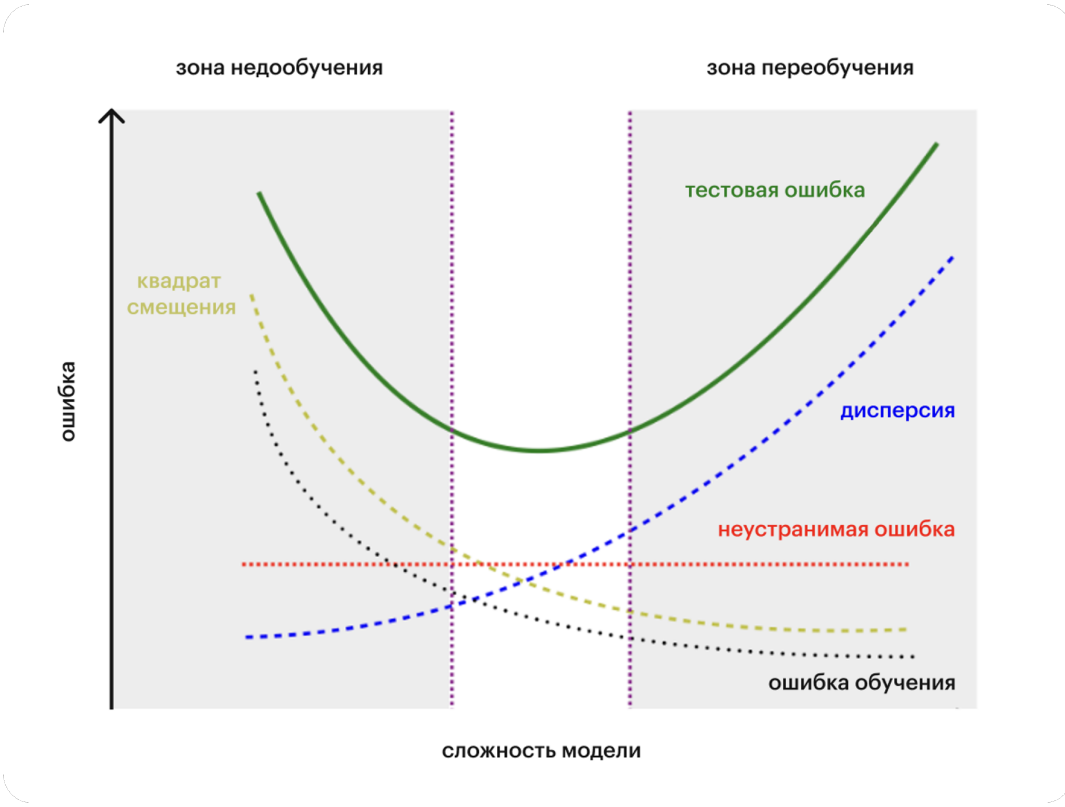


Рис. 7. Зависимость (квадратичного) смещения, дисперсии и общей тестовой ошибки от сложности модели

Объяснение этого феномена — на рисунке 7. На горизонтальной оси изображена сложность модели (например, количество параметров или степеней свободы), на вертикальной оси — значения функций. С увеличением сложности модели обучающая ошибка уменьшается и в итоге достигает нуля (как, например, в случае с интерполирующей функцией высокой сложности). Но тестовая ошибка (EPE) имеет U-образную форму и начинает повышаться начиная с какого-то значения сложности модели.

Причина в разном поведении трёх слагаемых:

- смещение с ростом сложности падает;
- дисперсия растёт;
- неустранимая ошибка не меняется вовсе. Она зависит от данных, а не от модели.

INTERVIEW KIT

Сначала уменьшение квадратичного смещения перекрывает рост дисперсии, и общая тестовая ошибка падает. Но дальше скорость сокращения смещения всё уменьшается, а скорость роста дисперсии увеличивается. В какой-то момент общая тестовая ошибка начинает расти. В машинном обучении это называется **bias-variance tradeoff**.

Слишком жёсткая модель (например, прямая) не гнётся под форму данных и промахивается примерно одинаково на любой выборке — это большое **смещение**.

Слишком гибкая (кривая, проходящая через каждую точку) подстраивается под случайные точки и на новой выборке рисует совсем другую линию — это большой **разброс**. Тестовая ошибка минимальна там, где модель гибкая ровно настолько, чтобы поймать форму зависимости, но не шум.

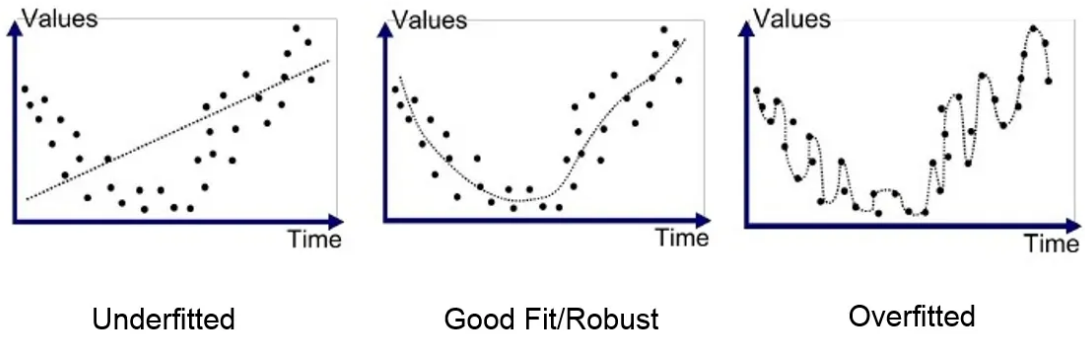


Рис. 8. Недообучение, оптимальное предсказание и переобучение на выборке. Источник

Переобучение

Разберём, что произойдёт, если при построении модели пренебречь анализом оптимальной сложности. Если переборщить со сложностью алгоритма, мы наткнёмся на переобучение: на рисунке 8 это правая картинка, а на рисунке 7 — правая часть графика (overfitting zone).

ОПРЕДЕЛЕНИЕ

Переобучение (overfitting) — состояние, при котором разброс растёт настолько, что перекрывает выигрыш от низкого смещения, и тестовая ошибка идёт вверх.

Обратная ситуация: подбираем слишком ригидный и недостаточно гибкий алгоритм. Это называется недообучением.

ОПРЕДЕЛЕНИЕ

Недообучение (underfitting) — состояние, при котором модель не может уловить все тенденции в данных (низкий разброс, высокая смещённость — underfitting zone).

Пере- и недообучение приводят к увеличению тестовой ошибки, а потому этих явлений стоит избегать. Обычно проблема переобучения стоит перед аналитиками острее, так как её сложнее обнаружить и избежать.

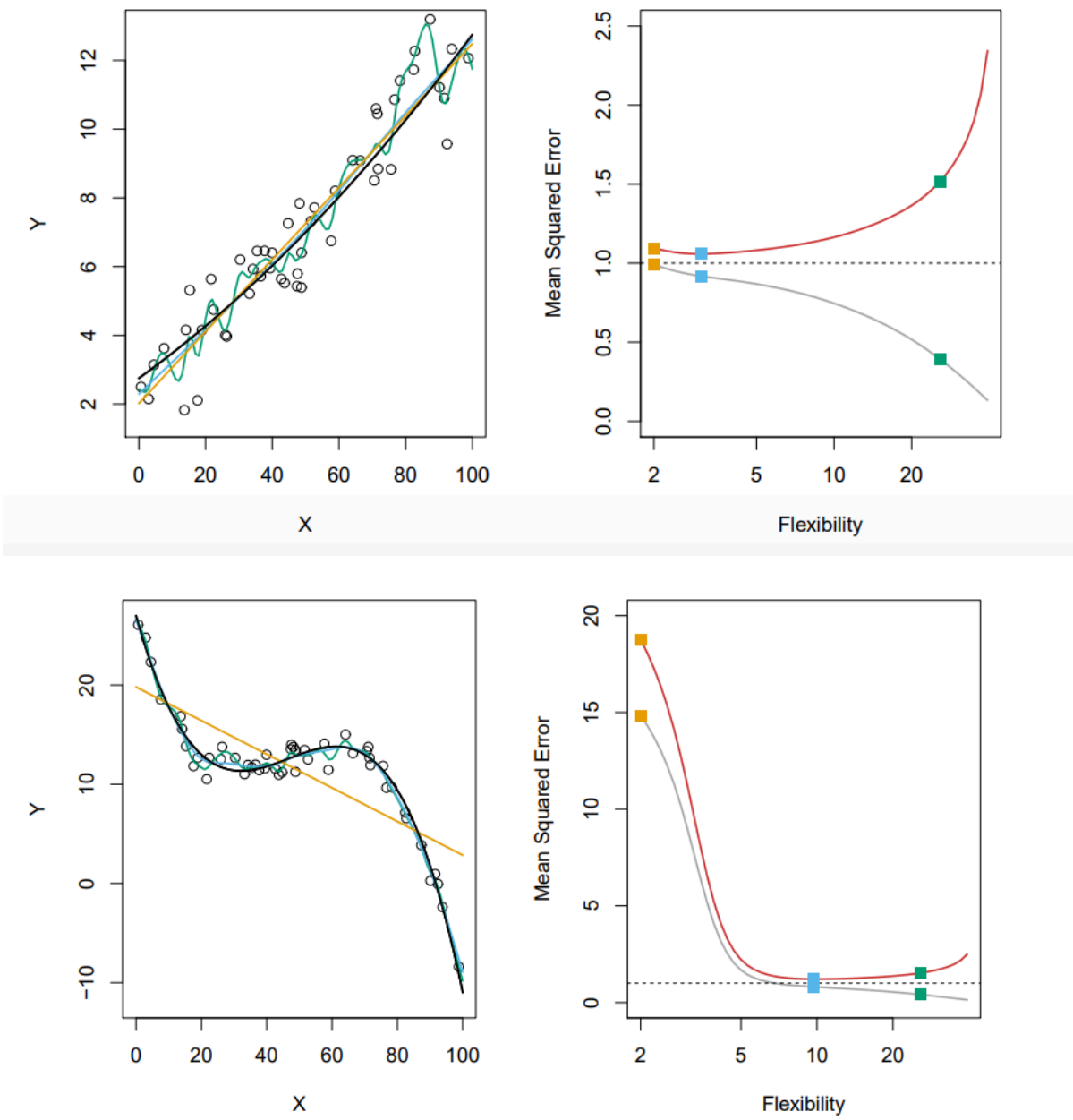


Рис. 9. Выбор оптимальной сложности зависит от распределения данных. Красная кривая — функция тестовой ошибки, серая кривая — функция обучающей ошибки. Три точки разных цветов обозначают три модели разных сложностей.
Источник: ISL, Hastie, Tibshirani

Рассмотрим рисунок 9. Оптимальная сложность модели — это не какое-то фиксированное значение, она зависит от самих данных. Если данные линейно зависимые (совместное распределение $Pr(X, Y)$), то лучше выбрать линейную модель, потому что увеличение сложности на таких данных быстро приведёт к переобучению. Если же данные распределены нелинейно, линейный алгоритм приведёт к недообучению, и необходимо будет обучение более комплексной и флексибильной модели. Поэтому перед проведением ML-анализа нужно смотреть на зависимости и распределения данных в выборке и проводить разведочный анализ данных (EDA).

Когда компромисс не работает

Итак, классическая картина такая: с ростом сложности смещение падает, разброс растёт, а тестовая ошибка идёт по U-образной кривой. Это рабочее правило, а не закон природы. У сильно перепараметризованных моделей — больших нейросетей, а иногда и ансамблей, тестовая ошибка после пика может пойти вниз ещё раз Belkin et al., 2019. Как это выглядит — на рисунке 10, а механика разобрана во врезке под ним.

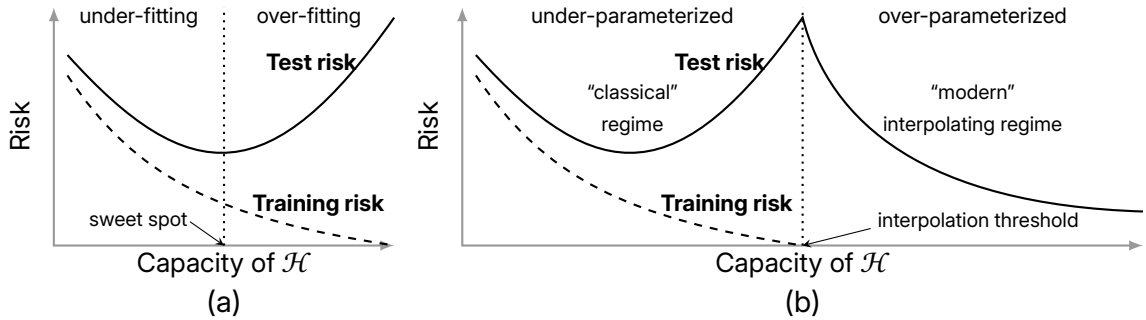


Рис. 10. Источник: Belkin, Hsu et al., 2019

ПРИМЕЧАНИЕ

Дополнительно: двойной спуск (double descent). Возьмём нейросеть и будем наращивать параметры не постепенно, а агрессивно. Сначала всё как обычно: ошибка падает, потом растёт. Пик приходится на порог интерполяции.

ОПРЕДЕЛЕНИЕ

Порог интерполяции (interpolation threshold) — момент, когда модель впервые способна пройти ровно через все обучающие точки.

ПРИМЕЧАНИЕ

Но если раздувать сеть дальше, тестовая ошибка снова начинает падать, иногда даже ниже классического sweet spot. Это и есть двойной спуск — его наблюдают не только у нейросетей, но и у ансамблей. Объясняется он через разброс. На самом пороге интерполяции способ пройти через все точки почти единственный: свободы у модели не остаётся, между точками кривая вынуждена резко изгибаться, а коэффициенты — расти. Отсюда пик разброса, а значит, и ошибки.

Когда параметров становится заметно больше, чем нужно для интерполяции, решений с нулевой ошибкой на обучении оказывается бесконечно много. И обучение, например, градиентный спуск — выбирает из них не любое решение, а самое «спокойное». То есть с наименьшей нормой весов. Между точками такая кривая гладкая, от выборки к выборке меняется слабо, разброс падает, и ошибка снижается второй раз.

Вывод: сверхпараметризация не отменяет переобучение, а даёт алгоритму выбрать более гладкое решение среди идеально подогнанных.

INTERVIEW KIT

Разложение ошибки на шум, смещение и разброс верно всегда, а вот у компромисса есть исключения: тестовая ошибка бывает не унимодальной, смещение и разброс меняются со сложностью не строго монотонно. А иногда рост сложности снижает сразу оба или, наоборот, поднимает оба. Практический вывод от этого не меняется: оптимальную сложность ищут по валидационной кривой, а не по формуле.

Классический пример

Подберём оптимальную сложность модели с минимальной тестовой ошибкой и наибольшей обобщающей и предсказательной способностью. Для этого разберём пример — покажем классический bias-variance tradeoff без «взрывного» роста параметров и крайних случаев.

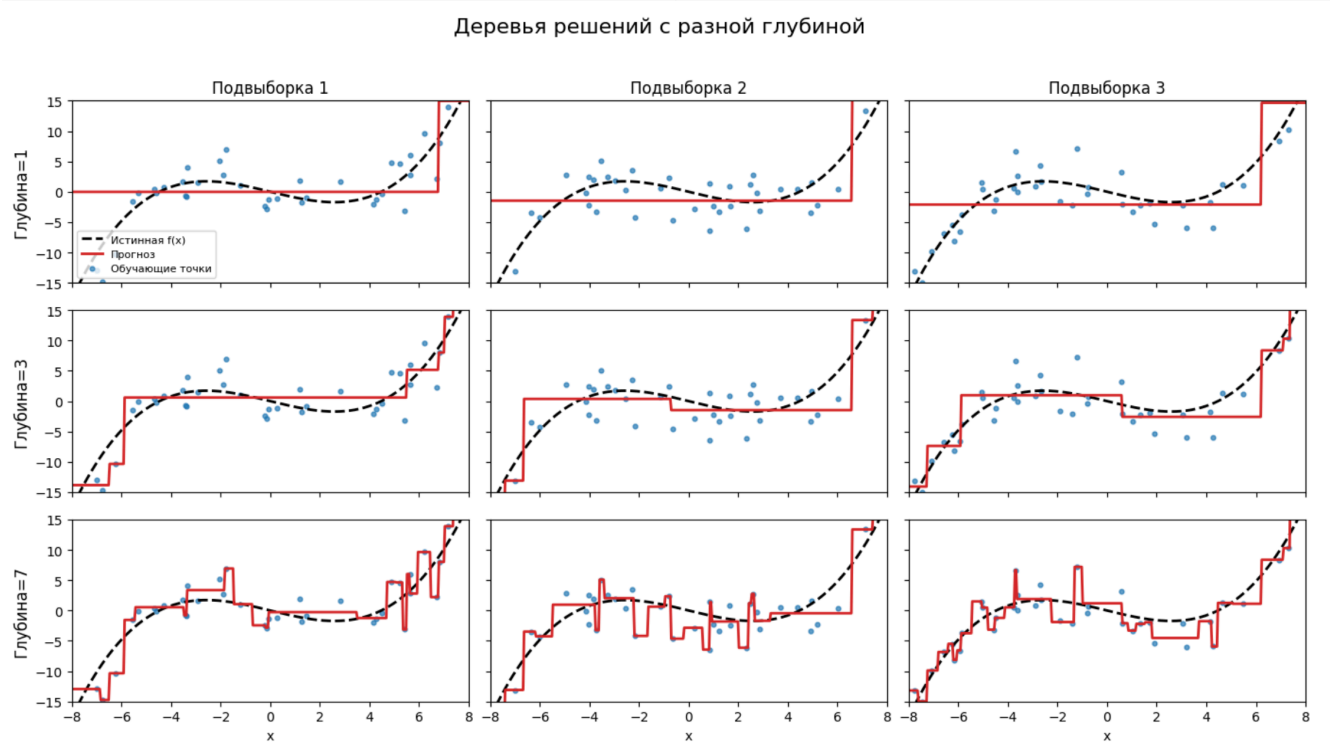


Рис. 11. Алгоритмы деревьев решений разной сложности (глубины) для трёх независимых выборок. Истинная зависимость кубическая

Предположим, что истинная зависимость в наших данных принимает кубическую форму:

$$f(x) = 0{,}05x^3 - x.$$

Тогда реальные значения будут включать в себя ошибку:

$$y_i = 0{,}05x_i^3 - x_i + \varepsilon, \qquad \varepsilon \sim \mathcal{N}\left(0,\ 2{,}5^2\right),$$

где ошибка распределена нормально с нулевым математическим ожиданием и дисперсией 6,25. Мы подобрали такую дисперсию ошибки, чтобы зависимость была неочевидной, но и не слишком размытой.

Сгенерируем три выборки по 40 наблюдений. Эти выборки используем в качестве тренировочных

для обучения деревьев решений — для каждой выборки обучим три дерева глубиной 1, 3 и 7.

Рассмотрим рисунок 11. Здесь тот же механизм можно увидеть наглядно: чем глубже деревья, тем сильнее расходятся между собой их предсказания на трёх разных выборках — это и есть разброс. Но деревья глубиной 1 слишком жёсткие, чтобы уловить кубическую зависимость. А оптимальной глубиной кажется значение 3–4. Подтвердим эту догадку.

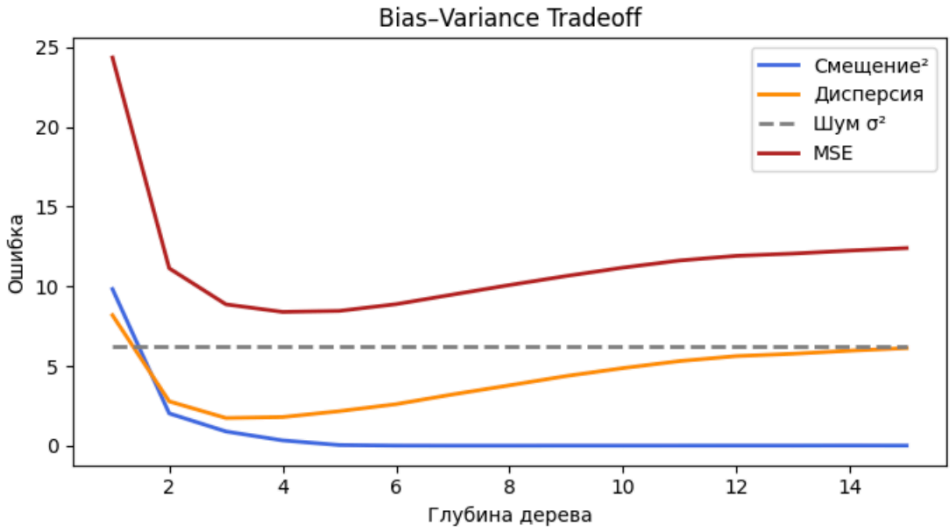


Рис. 12. Разложение тестовой MSE и bias-variance tradeoff на сгенерированных данных

Чтобы подтвердить догадку, проведём более тщательный анализ. Сгенерируем 400 обучающих выборок по 200 наблюдений каждая. Для каждого сета из 400 выборок обучим деревья решений глубиной от 1 до 15. Получится по 400 деревьев для каждого значения глубины. Для каждого значения глубины используем 400 обученных деревьев для расчёта смещения и дисперсии в 500 тестовых точках, распределённых равномерно на $[-8, 8]$.

Далее усредняем показатели по каждой тестовой точке и получаем финальный показатель смещения и дисперсии для определённой глубины дерева. По всем значениям глубины проводим такой же анализ. Неустранимым шумом в этом случае будет дисперсия ошибки $6{,}25$, одинаковая для всех глубин деревьев. Складываем три функции и получаем тестовый MSE на всех значениях глубины дерева.

Мы видим, что минимальный MSE достигается в значениях глубины 3–5. Недостаточная глубина ведёт к недообучению, а избыточная — к переобучению и сильному росту дисперсии.

Слева дисперсия сначала даже немного падает — это тот самый случай, о котором мы говорили выше: разброс растёт с гибкостью не строго монотонно. Дальше дисперсия при увеличении глубины стремится к уровню шума, а смещение — к нулю.

Причина в том, что очень глубокое дерево кладёт почти каждое наблюдение в отдельный лист. То есть предсказывает не $f(x_i)$, а само зашумлённое значение $y_i = f(x_i) + \varepsilon_i$. Поэтому в среднем по всем обучающим выборкам оно попадает точно в $f(x_i)$ — смещение стремится к нулю. Но от выборки к выборке его ответ изменяется ровно настолько, насколько изменяется шум. Поэтому дисперсия выходит на плато на уровне дисперсии шума, то есть 6,25.

1.4. Методы борьбы с переобучением

В реальной жизни не получится обучить модель на всех возможных выборках и посмотреть, как она себя ведёт «в среднем». Истинная зависимость тоже неизвестна, поэтому посчитать смещение и разброс по отдельности нельзя. Что же тогда делать на практике?

Все рабочие приёмы делятся на три группы — это контроль сложности модели, работа с данными и контроль качества. Разберём, на что именно каждая из них влияет.

INTERVIEW KIT

1. Контроль сложности модели. Бьём напрямую по разбросу: чем меньше у модели свободы, тем слабее она подстраивается под шум конкретной выборки. Плата — рост смещения, поэтому мера нужна во всём.

Для этого используют следующее.

- **Выбор модели и её архитектуры.** Иногда достаточно перейти с очень гибкой модели (глубокого дерева без ограничений, огромной нейросети) на более скромную: ограничить глубину, уменьшить число слоёв. Меньше параметров — ниже дисперсия.
- **Регуляризацию** — штраф за сложность прямо в функции потерь: L2 (ridge, weight decay) $\mathcal{L}(w) = \mathcal{L}_{\text{data}}(w) + \lambda\|w\|_2^2$ и L1 (lasso) $\mathcal{L}(w) = \mathcal{L}_{\text{data}}(w) + \lambda\|w\|_1$. Модель по-прежнему может быть гибкой, но за использование этой гибкости приходится «платить».
- **Сокращение размерности и отбор признаков.** PCA, UMAP, жадный отбор по важности. Меньше «мёртвых» признаков — меньше пространство, в котором модель может найти случайную закономерность.
- **Ансамблирование.** Усреднение множества моделей снижает разброс, почти не трогая смещение. Именно этому посвящена вторая часть лонгрида.

2. Работа с данными. Здесь мы меняем не модель, а то, на чём она учится. Разброс убывает потому, что выборка становится представительнее, а шума в ней — меньше.

Для этого используют следующее.

- **Увеличение объёма данных.** Чем больше качественных примеров, тем труднее их «запомнить», и тем ближе обучающая выборка к генеральной совокупности. Главный минус: данные дорогие, а иногда их просто неоткуда взять.
- **Аугментацию** — искусственное расширение выборки через преобразования, не меняющие ответ: повороты и сдвиги в компьютерном зрении, замена слов на синонимы в NLP, добавление шума в табличных данных.
- **Улучшение качества данных.** Чистка выбросов, исправление неверных меток, аккуратная работа с пропусками. Чем меньше в данных артефактов, тем меньше у модели поводов выучить несуществующую закономерность.
- **Выборка специальной структуры.** Осознанное формирование обучающего набора: добавление трудных примеров (hard negatives), балансировка классов, active learning — когда в разметку отправляют те объекты, в которых модель сомневается сильнее всего.

- **Использование внешних данных и знаний.** Transfer learning (дообучение модели, предобученной на большом корпусе) и multitask learning. По сути мы занимаем чужой опыт там, где своих данных мало.

3. Честная оценка качества. Эта группа не уменьшает переобучение сама по себе — она помогает его увидеть и вовремя остановиться. Диагностика простая: смотрим на разрыв между ошибкой на обучении и ошибкой на валидации. Если разрыв большой, а на обучении почти ноль — это разброс, модель переобучилась. А если обе ошибки высокие и близки друг к другу — это смещение, модель недообучилась. Без такой проверки первые две группы придётся применять вслепую.

Для этого используют следующее.

- **Разделение выборки и кросс-валидация.** Разбиение на train/validation/test и k-fold дают оценку ошибки на данных, которых модель не видела. По ней и подбирают сложность.
- **Ранняя остановка (early stopping).** Прерываем обучение, когда ошибка на валидации перестала падать, даже если на обучении она продолжает снижаться.
- **Дисциплина работы с тестом.** Тестовая выборка используется один раз, в самом конце. Если подбирать по ней гиперпараметры, она превращается в ещё одну валидационную и начинает завышать качество.

ПОДСКАЗКА

Как выбрать, с чего начать:

- если диагноз «высокий разброс» — работают группы 1 и 2: упростить модель, добавить регуляризацию, набрать данных, усреднить несколько моделей;
- если диагноз «высокое смещение» — всё наоборот: модель надо усложнять, а сдерживать её вредно.

1.5. Выводы по блоку

- Смещение и разброс по отдельности на реальных данных не измерить: истинная зависимость неизвестна, а обучающая выборка у нас одна. Судим о них косвенно — по разрыву между ошибкой на обучении и ошибкой на валидации.
- Контроль сложности — ограничение модели, регуляризация, отбор признаков — бьёт по разбросу, но обычно поднимает смещение. Поэтому нужна мера: цель не «зажать модель посильнее», а попасть в минимум тестовой ошибки. Ансамблирование стоит особняком: оно снижает разброс, не меняя смещения, — об этом в части 2.
- Работа с данными — больше примеров, аугментация, чистка разметки, перенос знаний — снижает разброс, не зажимая саму модель. Это единственная группа, которая делает выборку представительнее, а не модель скромнее.
- Честная оценка качества — отложенная выборка, кросс-валидация, ранняя остановка, дисциплина работы с тестом — переобучение не лечит, но позволяет его вовремя увидеть. Без неё первые две группы применяются вслепую.
- Оптимальная сложность — не универсальная константа: она зависит от данных, поэтому её не выбирают заранее, а подбирают на валидации или кросс-валидации. И только потом один раз

проверяют на тесте.

А теперь вернёмся к деревьям решений, чтобы изучить методы контроля сложности и предотвращения переобучения.

1.6. Переобучение деревьев решений

INTERVIEW KIT

Склонность деревьев к переобучению не случайна — она встроена в сам алгоритм и есть прямое следствие его непараметричности. У дерева нет ни заранее заданной формы зависимости, ни фиксированного числа параметров: число листьев определяется данными по ходу обучения. Поэтому ничто само по себе не мешает дереву расти, пока объекты не кончатся, — границу приходится устанавливать вручную.

На прошлой неделе мы уже видели это вживую: с ростом максимальной глубины решающая граница дерева распадалась на узкие полоски, гоняющиеся за отдельными шумными точками, точность на обучении доходила до 1,0, а на отложенной выборке сначала росла, а потом падала. Тогда мы просто зафиксировали картинку и договорились подбирать глубину по валидации.

Но теперь это можно объяснить так: неограниченное дерево кладёт почти каждое наблюдение в отдельный лист и предсказывает не истинное среднее $f(x_i)$, а конкретное зашумлённое y_i. Значит, ответ целиком определяется тем, какая выборка попалась, — почти вся ошибка сидит в разбросе. И лечить надо именно разброс: сознательно занизить гибкость дерева, доплатив за это смещением.

Способы сдержать дерево делятся на два класса — по тому, в какой момент происходит вмешательство.

1. **Pre-pruning-методы** (ранняя остановка) — не даём дереву вырасти лишним прямо во время обучения.
2. **Post-pruning-методы** (стрижка) — растим дерево до конца, а потом срезаем ветки, которые не окупают свой размер.

Разберём pre- и post-pruning методы подробнее.

1.7. Pre-pruning-методы

Мы не даём дереву вырасти слишком большим с самого начала, накладывая на обучение определённые ограничения. Эти ограничения — гиперпараметры, которые нужно регулировать до начала обучения модели.

Ограничения могут быть заданы в виде гиперпараметров библиотеки Python (`sklearn.tree.DecisionTree*`). Их проверяют по следующему алгоритму.

1. При попадании в какой-либо узел проверить условие.
2. Если условие соблюдается — объявить вершину терминальной (листом).
3. Проверить условие на других узлах.

Условия

- Глубина дерева — `max_depth`. Например, при `max_depth = 10`: если глубина во всех узлах-претендентах достигла 10, останавливаем рост дерева и объявляем алгоритм обученным.
- Минимальное число объектов в узле — `min_samples_split`, `min_samples_leaf`.
- Минимальное уменьшение ошибки при делении —`min_impurity_decrease`. Если лучшее возможное деление узла уменьшает ошибку меньше, чем на заданный порог, делить дальше не имеет смысла — вершина объявляется листом.
- Максимальное количество листьев — `max_leaf_nodes`. Считает общее количество листьев в дереве и при превышении завершает обучение. При применении этого ограничения важно, в каком порядке обучаем дерево (level-wise/leaf-wise tree growth).
- Максимальное количество признаков при поиске лучшего разбиения — `max_features`. Особенно полезно будет в дальнейшем при обучении ансамбля случайных лесов.

Методы раннего сдерживания переобучения просты в применении. Ранняя остановка оценивает сплит сам по себе. Бывает, что сплит почти ничего не даёт, но открывает дорогу сильному следующему — как в XOR. Ограничение отсечёт слабый сплит, и дерево недообучится

1.8. Post-pruning-методы

Этот класс методов предполагает построение неограниченного дерева, которое затем «стригут» до состояния оптимального поддерева. Это помогает:

- увеличить смещение оценок;
- уменьшить дисперсию предсказаний, понижая общую тестовую ошибку.

В сравнении с pre-pruning-методами, методы стрижки деревьев дают гораздо большую гибкость в выборе оптимального решения и ведут к меньшему риску недообучения. Но у методов стрижки есть два минуса — они:

- сложнее в реализации,
- дольше считают.

Поэтому перед применением метода нужно оценить, хватит ли мощностей для его работы. Так в некоторых случаях лучше применить более простой pre-pruning.

В идеале стоит перебрать все поддеревья и выбрать дерево с наименьшей валидационной ошибкой. Но поддеревьев слишком много: гонять кросс-валидацию для каждого при большом числе листьев невозможно. Поэтому берут жадный алгоритм — cost-complexity pruning.

ОПРЕДЕЛЕНИЕ

cost-complexity pruning (он же weakest link pruning) — это алгоритм, который вместо экспоненциального перебора строит не все поддеревья, а цепочку кандидатов длиной в число узлов дерева. Далее валидационную ошибку сравнивают только у них.

Разберём дерево регрессии при квадратичном лоссе. Обозначения простые:

- T — текущее дерево (в начале это T_0, полностью обученное);
- $|T|$ — число его листьев;
- R_j — j-й лист;
- $|R_j|$ — число объектов в нём;
- $H_j(T)$ — знакомая по неделе 6 мера неопределённости (impurity) внутри листа, то есть средний квадрат отклонения ответов в листе от среднего по этому листу;
- $|R_j|\,H_j(T)$ — суммарная ошибка листа, и именно из таких слагаемых собирается функция стоимости ниже;
- $T \subset T_0$ — любое дерево, которое получается из T_0 схлопыванием каких-то внутренних узлов в листья.

INTERVIEW KIT

Определим критерий cost complexity (функцию стоимости):

$$C_\alpha(T) = \sum_{j=1}^{|T|} |R_j| H_j(T) + \alpha |T|,$$

Первое слагаемое — ошибка дерева на обучении, второе — штраф за число листьев. Гиперпараметр $\alpha \geq 0$ задаёт компромисс между размером дерева и качеством подгонки: при $\alpha = 0$ выигрывает полное дерево T_0. А чем больше α, тем короче лучшее поддерево. Для каждого α существует ровно одно наименьшее поддерево T_α, минимизирующее $C_\alpha(T)$, — его и берут.

Считаем α «платой за каждый лист» — налогом на размер дерева. Разбиение узла уменьшает ошибку, но добавляет листья, и штраф $\alpha|T|$ растёт. Поэтому выживают только те ветки, что уменьшают ошибку сильнее, чем стоит их содержание. Чем больше α, тем дороже листья и тем короче подрезанное дерево. Это ровно та же идея регуляризации, что и штраф $\lambda\|w\|$ у линейных моделей, только для деревьев.

Перебирать α вслепую не нужно: алгоритм сам строит цепочку поддеревьев — от полного дерева до одного корня, — и у каждого звена цепочки есть своё α. Оптимальное $\hat{\alpha}$ выбирают по пятикратной или десятикратной кросс-валидации, а ответом становится поддерево $T_{\hat{\alpha}}$. В `sklearn` за это отвечает гиперпараметр `ccp_alpha`.

ПРИМЕЧАНИЕ

Дополнительно: как строится цепочка поддеревьев (weakest link).

1. На каждом шаге срезаем «самое слабое звено» — ветвь, которая хуже всех окупает свой размер.
2. Для каждого внутреннего узла считаем отношение:

$$g = \frac{\text{прирост ошибки от схлопывания узла}}{\text{число удаляемых при этом листьев} - 1}.$$

Затем схлопываем узел с наименьшим g. Делить на число листьев обязательно: крупная ветвь почти всегда даёт больший прирост ошибки просто потому, что она

> крупная. А нас интересует цена за один лист.
>
> Значение g у срезанного узла — это ровно то α, при котором схлопывание становится выгодным. Поэтому шаги алгоритма сами собой упорядочены по возрастанию α, а цепочка тянется до дерева из одной вершины.

1.9. Выводы первой части

- Глубокое неограниченное дерево — учебный пример низкого смещения при высоком разбросе.
- Глубокое неограниченное дерево почти дословно запоминает обучающую выборку, поэтому в среднем по выборкам попадает близко к истинной зависимости. Но от выборки к выборке его ответы сильно гуляют.
- Первый способ это поправить — ограничить само дерево. Pre-pruning не даёт ему разрастись во время обучения, post-pruning стрижёт уже выросшее. Оба приёма сбивают разброс, но платят за это ростом смещения, и одиночное подрезанное дерево остаётся середняком по точности.

Во второй части лонгрида «Ансамбли» разберём другой подход — не ограничивать дерево, а усреднить предсказания сразу многих деревьев. Так разброс снижается, почти не задевая смещение. На этой идее и построены ансамбли.

2. АНСАМБЛИ

Теперь перейдём к ансамблям и разберём, как они устроены.

2.1. Почему одиночной модели недостаточно

Неограниченное дерево даёт низкое смещение, но платит за это большей дисперсией. На прошлой неделе мы уже сталкивались с ней под другим именем: нестабильность деревьев. Достаточно сдвинуть в обучающей выборке одно-единственное наблюдение, и кусок ответа переписывается. Так, на рисунке 13 слева дерево, обученное на исходной выборке. А справа — на той же выборке со сдвинутой точкой. Почти вся ступенчатая кривая совпадает, но вокруг этой точки предсказание проваливается с $+1{,}0$ до $-1{,}0$.

Тогда мы уже называли это высокой дисперсией, но не могли объяснить, почему дисперсия — это плохо. Теперь можем: после первой части лонгрида мы знаем, что разброс входит в тестовую ошибку отдельным слагаемым. Именно он повышает ошибку на новых данных, каким бы низким ни было смещение дерева.

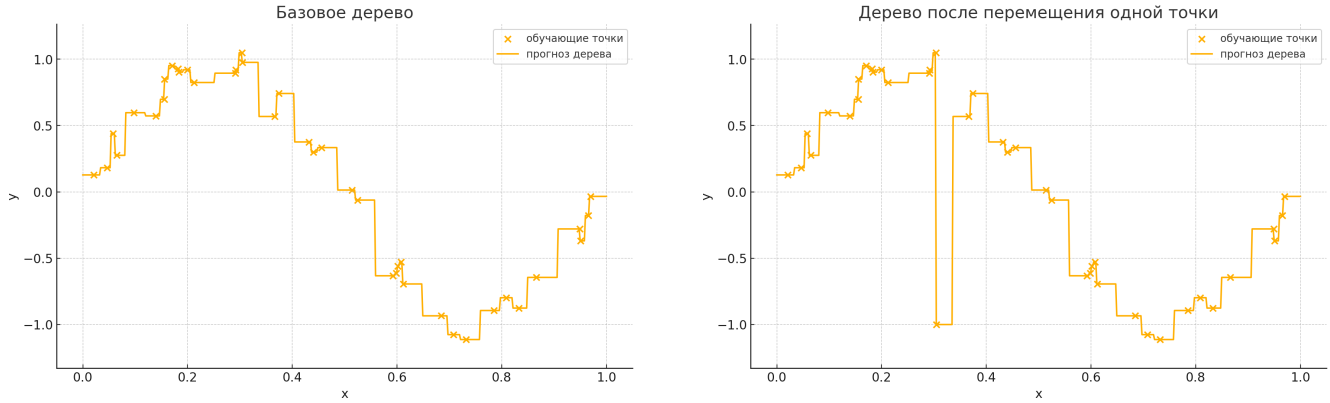


Рис. 13. Два дерева регрессии: левое обучено на изначальной выборке, а правое — на той же выборке, но с одним изменённым наблюдением. Экстремальная дисперсия деревьев решений резко повышает их тестовую ошибку

Ограничения в виде гиперпараметров этот разброс сдерживают, но платой за это становится рост смещения. Даже аккуратно постриженное дерево остаётся чувствительным к шуму и выбросам и даёт среднюю точность в задачах классификации и регрессии. Из-за этого одиночные деревья используют в ситуациях, когда критична интерпретируемость и бизнес готов пожертвовать точностью ради понятных и прозрачных моделей.

Проблема переобучения характерна и для других гибких моделей — полиномиальной регрессии и нейросетей с большим числом параметров. При этом слишком жёсткие модели, наоборот, могут недообучаться. В итоге одиночная модель часто не даёт оптимального баланса между смещением и дисперсией.

Здесь на помощь приходят ансамбли: они объединяют десятки, сотни и даже тысячи разных (или однотипных) моделей, чтобы компенсировать слабые стороны отдельных алгоритмов. При удачном подборе базисов ансамбль даёт более устойчивые предсказания, чем одиночная модель. Но гарантий здесь нет: если базисы сильно схожи, или один из них заметно сильнее остальных, ансамбль может и не обойти лучший базис.

Одна голова — хорошо, а тысячи — ещё лучше.

2.2. Ансамбли как лекарство

Само дерево менять не придётся: оставим его гибким, с низким смещением. А разброс снизим не внутри модели, а снаружи — через усреднение многих таких деревьев. Здесь разница принципиальна: стрижка из части 1 снижает разброс повышая смещение, а усреднение снижает разброс, не меняя смещения.

ОПРЕДЕЛЕНИЕ

Ансамбль (англ. ensemble learning) — это парадигма машинного обучения, в которой несколько базовых моделей обучаются на одной задаче, а их предсказания объединяются (например, голосованием или усреднением) для повышения точности, надёжности и устойчивости итогового решения.

Ансамбль можно сравнить с командой супергероев: каждая модель использует свою уникальную «силу», чтобы вместе эффективнее победить злодея-ошибку (см. рисунок 14).

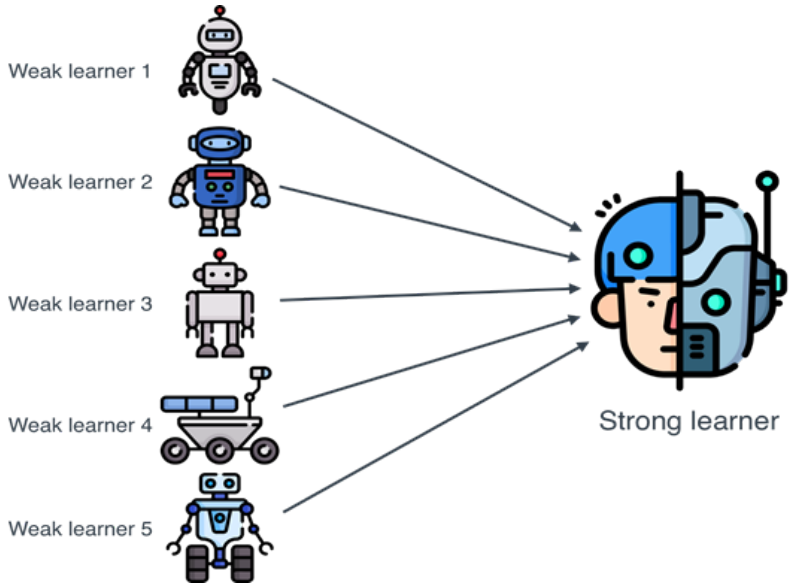


Рис. 14. Ансамбль комбинирует мощи каждой отдельной базовой модели. Источник

В общем случае ансамбль алгоритмов (или композиция) имеет вид

$$a(x) = F(b_1(x), b_2(x), \dots, b_K(x)).$$

Здесь:

- b_k — базовые алгоритмы (по отдельности неустойчивые: низкое смещение, высокая дисперсия);
- F — метаалгоритм, который агрегирует предсказания K базовых алгоритмов, чтобы получить более качественный общий алгоритм.

Другими словами.

1. В каждой точке $X = x$ мы рассчитываем предсказания базовых моделей $b_1(x), b_2(x), \dots, b_K(x)$, получая вектор из ответов размерностью K.
2. С помощью агрегирующей функции конвертируем этот вектор в финальное предсказание для каждой из точек $X = x$.

Главная причина, по которой ансамбли работают, — это статистика. Если базисы почти не смещены и слабо связаны между собой, усреднение их предсказаний снижает дисперсию итоговой оценки. А вместе с ней падает и тестовая ошибка. Именно эту механику мы дальше разберём подробно: сначала на числах, а потом и формулой.

ПРИМЕЧАНИЕ

Дополнительно: ещё две причины собирать ансамбль

Вычислительная — обучение одной тяжёлой модели упирается в ресурсы, а ансамбль распадается на независимые подзадачи. Базовые модели можно обучать параллельно — на разных ядрах или машинах.

Функциональная — набор простых моделей вместе описывает более сложные зависимости, чем каждая из них по отдельности. Примерно как из нескольких пороговых классификаторов складывается ломаная граница любой формы.

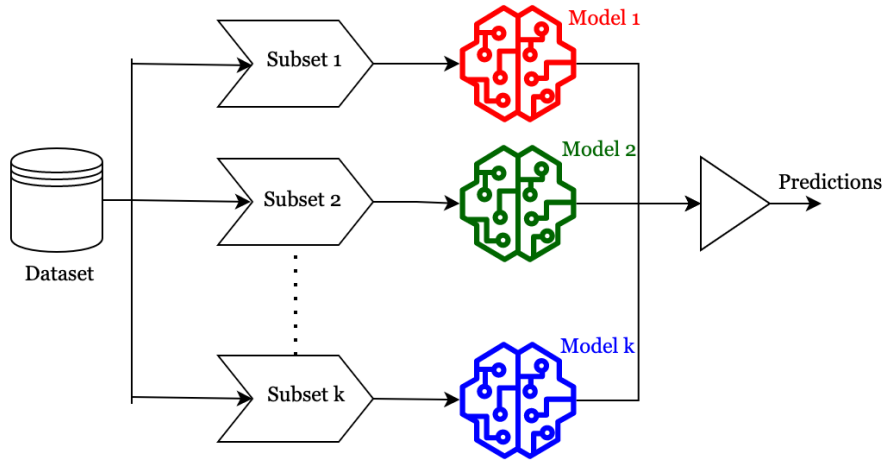


Рис. 15. Пример параллельного ансамбля из K базовых моделей, которые отличаются друг от друга и обучены на разных подвыборках. Итоговое предсказание — вывод агрегирующей функции. Источник

2.3. Как обучается и работает ансамбль

Любую задачу по обучению ансамбля можно разложить на два этапа.

Этап 1. Обучение базовых алгоритмов $b_1(x), b_2(x), \ldots, b_K(x)$.

1. Работа с данными — определяем, на каких данных будет обучаться каждая модель. Есть несколько методов:
   - обучать каждую модель на одних и тех же объектах;
   - бутстрэп (англ. bootstrap) из оригинальной выборки (с возвращением);
   - пастинг (англ. pasting) из оригинальной выборки (без возвращения);
   - Random Subspace — каждая модель обучается на всех объектах, но лишь на случайной части признаков.
2. Выбор базовых алгоритмов (деревья решений, нейросети, kNN, SVM, линейные модели (реже)).
   От выбора алгоритма зависит тип ансамбля:
   - гомогенный — одинаковые базовые алгоритмы;
   - гетерогенный — разные базовые алгоритмы.
3. Подбор гиперпараметров для базовых алгоритмов.
4. Выбор структуры, диктующей этапы построения ансамбля:
   - параллельная структура — все базисы обучаются одновременно и независимо;
   - последовательная структура — базисы обучаются один за другим.

Этап 2. Выбор агрегирующей функции (метаалгоритма) F, которая комбинирует предсказания базовых моделей в одно итоговое предсказание. Агрегация может быть:

- фиксированной — правило задано заранее, ничего дополнительно не обучаем поверх базовых моделей. Примеры: среднее арифметическое (регрессия), мода (классификация);
- адаптивной — есть отдельный процесс подстройки параметров агрегатора, зависящий от данных. Пример: сложный метаалгоритм, который обучается на предсказаниях базовых моделей.

После того как ансамбль обучен, мы используем его для предсказания ответов на тестовых данных. Для тестовой точки $X = x_{\text{test}}$ находим предсказание ответа для этой точки от каждой базовой функции. Далее все предсказания агрегируются метаалгоритмом, и получается итоговая оценка $\hat{y}_{\text{test}}$ в точке x_{test}.

INTERVIEW KIT

Каждый ансамблевый метод — это отдельная комбинация базовых моделей, данных и способов агрегирования. И таких комбинаций может быть много. Разнообразие нужно не просто так: вариативность — критически важное свойство для эффективности ансамбля. Именно она помогает моделям дополнять друг друга и снижать общую ошибку.

ПРИМЕЧАНИЕ

Пример. Пусть есть:

- 10 наблюдений и задача бинарной классификации;
- 3 модели с одинаковой точностью, но с разной степенью схожести ошибок. И для каждой запишем, где она угадала. Здесь:
  - 1 — модель ответила верно;
  - 0 — ошиблась;
- у каждой модели ровно 5 верных ответов из 10 (точность каждой — 50%).

Комбинируем предсказания по правилу большинства: итоговый ответ на объекте — это мода трёх ответов (агрегатор — мода).

Случай 1. Модели почти одинаковые.

- `1111100000` — модель 1;
- `1111100000` — модель 2;
- `1111010000` — модель 3.

Мода: `1111100000` — точность ансамбля снова 50%. Модели ошибаются на одних и тех же объектах, поэтому голосование ничего не улучшает.

Случай 2. Предсказания разные, но ошибки всё равно совпадают.

- `1111100000` — модель 1;
- `1111010000` — модель 2;
- `1111001000` — модель 3.

Мода: `1111000000` — точность 40%, стало хуже. Там, где ошиблись две модели из трёх, большинство «утянуло» общий ответ в ошибку.

Случай 3. Модели разносторонние — их ошибки почти не пересекаются.

- `1111100000` — модель 1;
- `1110011000` — модель 2;
- `1001111000` — модель 3.

Мода: `1111111000` — точность 70%. Каждая модель ошибается на «своих» объектах, а большинство исправляет промахи одиночек.

Обрати внимание: три модели с точностью 50% нельзя скомбинировать в 100% и даже в 80% — у голосования большинством трёх таких моделей есть потолок 70%. Весь выигрыш ансамбля берётся не из числа моделей, а из того, что они ошибаются на разных объектах.

Суть ансамбля — объединить максимально разносторонние базисные модели, чтобы их сильные стороны усиливали друг друга и компенсировали индивидуальные слабости. Это помогает получить

более надёжное и точное предсказание, чем у любой отдельной модели (см. рисунок 16).

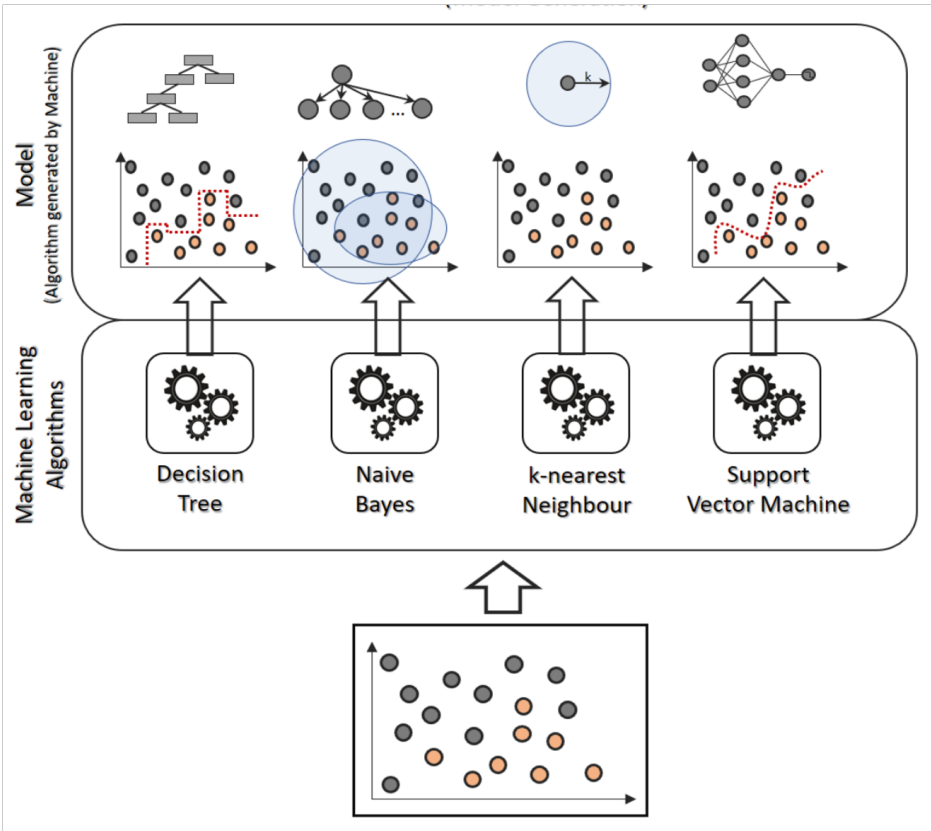


Рис. 16. Цель — сконструировать максимально разнообразные базисные модели, которые бы перекрывали ошибки друг друга. Источник

ПОДСКАЗКА

Сила ансамбля заключается не во множестве его моделей, а в том, чтобы они допускали ошибки на разных объектах, создавая ценное разнообразие.

Как раз для внесения разнообразия и были придуманы схемы работы с данными и выбора моделей, а также различные структуры обучения и агрегирующие функции.

2.4. Математика за разнообразием

Теперь объясним эту «необходимость в разнообразии» с математической точки зрения. Для этого посмотрим, как разнообразие моделей влияет на смещение, дисперсию и тестовую ошибку ансамбля. Спойлер: в параллельных ансамблях разнообразие базисов не влияет на смещение, но может сильно повлиять на дисперсию всего ансамбля.

ВАЖНО

Последующие вычисления относятся только к параллельным ансамблям (например, Bagging, Pasting, случайные леса), которые мы будем разбирать на этой неделе. В задачах последовательных ансамблей (Boosting) — немного другая логика. К бустинговым методам мы перейдём в лонгридах следующих недель.

Определим задачу регрессии.

Существует истинная целевая функция регрессии $f(x)$ (или истинное условное математическое ожидание $\mathbb{E}[Y \mid X = x]$) в точке $X = x$.

Мы хотим использовать ансамбль моделей $b_k(x)$, чтобы оценить значения истинной функции в точке x.

Возьмём за агрегирующую функцию ансамбля среднее арифметическое. Таким образом, у нас получается ансамбль вида

$$a(x) = \frac{1}{K}(b_1(x) + b_2(x) + \cdots + b_K(x)).$$

Предположим, что оценки базисов $b_1(x), b_2(x), \ldots, b_K(x)$ одинаково распределённые (англ. identically distributed, ID). Это значит, что в точке $X = x$ у оценок всех базисов (как случайные переменные) одинаковое вероятностное распределение, и следовательно

$$\forall\, k, l: \quad \mathbb{E}\big[b_k(x)\big] = \mathbb{E}\big[b_l(x)\big] = \mu(x) \qquad \text{и} \qquad \mathrm{Var}\big[b_k(x)\big] = \mathrm{Var}\big[b_l(x)\big] = \sigma^2(x),$$

где $\sigma^2(x)$ — дисперсия одного базиса (не путать с σ_ε^2 — неустранимым шумом из первой части).

Смещение

Среднее арифметическое — несмещённая оценка математического ожидания:

$$\mathbb{E}\big[a(x)\big] = \mathbb{E}\left[\frac{1}{K}\sum_{k=1}^{K} b_k(x)\right] = \frac{1}{K}\sum_{k=1}^{K}\mathbb{E}\big[b_k(x)\big] = \frac{1}{K}\cdot K\,\mu(x) = \mu(x).$$

А это значит:

$$\mathrm{Bias}\big[a(x)\big] = \mathbb{E}\big[a(x)\big] - f(x) = \mu(x) - f(x) = \mathrm{Bias}\big[b_k(x)\big].$$

INTERVIEW KIT

Вывод: при усреднении одинаково распределённых моделей смещение не меняется.

Интуитивно: усреднение ответов базовых алгоритмов не способно убрать смещение, присущее этим базовым алгоритмам. Например, если все базисы завышают ответ на 2, то и среднее завысит на 2.

Дисперсия

Пусть попарные ковариации всех пар базисов одинаковы:

$$\forall\, k \neq l\ :\ \ \mathrm{Cov}\big(b_k(x), b_l(x)\big) = c(x).$$

Тогда коэффициент корреляции Пирсона между оценками базисов равен

$$\rho(x) = \frac{c(x)}{\sigma^2(x)}.$$

Тогда:

$$\begin{aligned}\mathrm{Var}[a(x)] &= \frac{1}{K^2}\Big(\sum_{k=1}^{K}\mathrm{Var}\big[b_k(x)\big] + \sum_{k\neq l}\mathrm{Cov}\big(b_k(x), b_l(x)\big)\Big) = \\ &= \frac{1}{K^2}\big(K\,\sigma^2(x) + K(K-1)\,c(x)\big) = \frac{\sigma^2(x)}{K}\big(1 + (K-1)\rho(x)\big) = \\ &= \rho(x)\sigma^2(x) + \frac{1-\rho(x)}{K}\sigma^2(x).\end{aligned}$$

Откуда мы делаем вывод:

$$\text{если}\quad \rho = 1 \implies \mathrm{Var}[a(x)] = \sigma^2(x), \text{если}\quad \rho = 0 \implies \mathrm{Var}[a(x)] = \frac{\sigma^2(x)}{K}.$$

INTERVIEW KIT

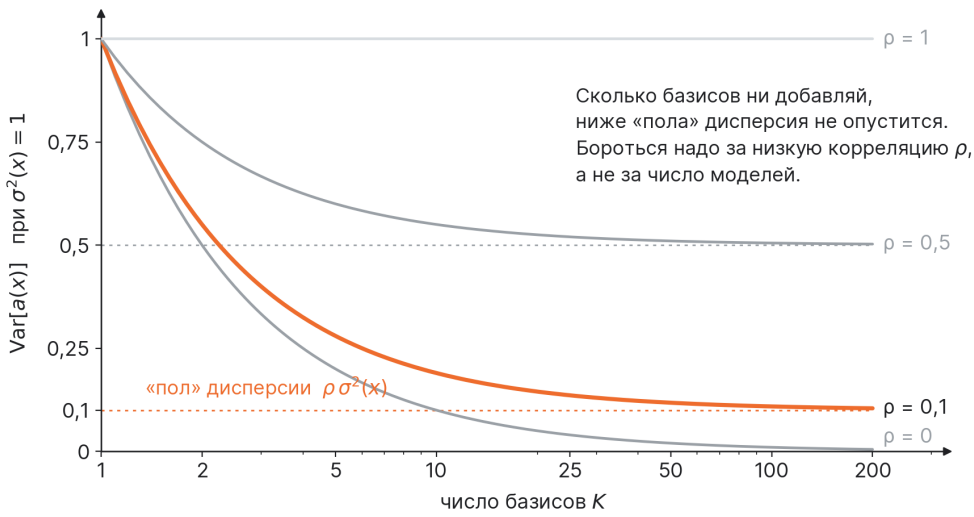


Рис. 17. Как убывает дисперсия ансамбля с ростом числа базисов K при разной корреляции между ними. При $\rho > 0$ кривая упирается в «пол» $\rho\,\sigma^2(x)$

Чем ниже корреляция между базисами, тем меньше итоговая дисперсия ансамбля.

- При идеальной корреляции (когда $\rho = 1$) эффекта от ансамбля нет, модели полностью копируют друг друга, и их усреднение не даёт никакой практической выгоды.
- При полной некоррелированности моделей (когда $\rho = 0$) эффект от ансамбля максимален, как в на рисунке 14, каждая базовая модель выступает супергероем с уникальными способностями. А вместе эти супергерои становятся непобедимыми. В этом идеальном случае дисперсия ансамбля в K раз меньше дисперсии каждого отдельного базиса.

ПРИМЕЧАНИЕ

Пример. Возьмём $K = 100$ базисов и дисперсию одного базиса $\sigma^2(x) = 1$. Подставляем в формулу $\mathrm{Var}[a] = \rho\,\sigma^2(x) + \dfrac{1-\rho}{K}\,\sigma^2(x)$:

- $\rho = 0$ (некоррелированы): $\mathrm{Var}[a] = 0{,}01$ — в 100 раз меньше, чем у одного базиса;
- $\rho = 0{,}1$: $\mathrm{Var}[a] = 0{,}1 + 0{,}009 = 0{,}109$ — уже только в ≈ 9 раз меньше;
- $\rho = 0{,}5$: $\mathrm{Var}[a] = 0{,}5 + 0{,}005 = 0{,}505$ — почти в 2 раза меньше;
- $\rho = 1$ (копии): $\mathrm{Var}[a] = 1$ — никакого выигрыша.

Даже сотня моделей почти не помогает, если они сильно скоррелированы: слагаемое $\rho\,\sigma^2(x)$ — это «пол», ниже которого дисперсию не опустить, сколько базисов ни добавляй. Поэтому вся борьба идёт за низкую корреляцию ρ, а не за число моделей.

В итоге это сильно понижает тестовую ошибку ансамбля, повышая его обобщающую способность.

Так как основная цель параллельных ансамблей — это понижение общей дисперсии, этот класс алгоритмов хорошо работает только при гибких базисах, которые в среднем почти не ошибаются, но выдают очень разбросанные оценки (ансамбль сохраняет низкий bias и сильно снижает высокий variance модели). Поэтому дерево решений — один из самых распространённых базисов для ансамблей.
С другой стороны линейные модели в параллельных ансамблях почти не используются — ансамбль не может снизить высокое смещение, а дисперсия у линейных моделей и так низкая.

ПРИМЕЧАНИЕ

Работу ансамбля объясняет статистика. Если базовые модели одинаково распределены и попарно некоррелированы, дисперсия ансамбля равна $\sigma^2(x)/K$ и стремится к нулю при росте K.

Здесь нужна именно некоррелированность, а не независимость — из $\rho = 0$ независимость не следует, и всему выводу выше она не требуется.
Для независимых величин это частный случай закона больших чисел.

В теории ансамбли обещают заметное улучшение качества моделей, но на практике базовые алгоритмы одного ансамбля:

- решают одну задачу;
- обучаются на похожих (или вообще одинаковых) данных;
- настраиваются на один целевой вектор;
- могут быть из одного семейства моделей (или из нескольких, но всё равно небольшого числа).

Всё это приводит к тому, что корреляция между базисами бывает нулевой очень редко: на практике базовые алгоритмы часто скоррелированы. А увеличение корреляции между базисами ведёт к уменьшению положительного эффекта от ансамбля, так как дисперсия уменьшается не в K раз, а меньше.

Таким образом, задача следующая.

1. Сделать ансамбль максимально разнообразным, чтобы усилить эффект от падающей дисперсии (соответственно, понизить тестовую ошибку ансамбля).
2. Сделать саму композицию алгоритмов более устойчивой к изменениям в обучающих данных.

Далее рассмотрим ключевые ансамблевые техники, каждая из которых по-своему усиливает внутреннее разнообразие моделей.

3. BAGGING И ЕГО СЕМЕЙСТВО

Начнём с самого базового параллельного ансамбля — бэггинга. На его идее (обучать модели на разных случайных подвыборках и усреднять их ответы) построены и случайный лес, и его родственники. Именно бэггинг нагляднее всего показывает, как разнообразие базисов снижает дисперсию на практике.

3.1. Bagging

Рассмотрим два этапа построения ансамбля на примере бэггинга — обучение базисов и выбор агрегатора.

Этап 1. Выбор агрегатора

Начнём с подбора агрегирующей функции в бэггинге.

- База для регрессии — среднее арифметическое:

$$a(x) = \frac{1}{K}(b_1(x) + b_2(x) + \cdots + b_K(x)).$$

- Модификация для регрессии — иногда может использоваться медиана:

$$a(x) = \text{median}(b_1(x), b_2(x), \ldots, b_K(x)).$$

- База для классификации — когда нужно для каждой точки $X = x$ предсказать конкретный класс, можно взять класс, который предсказывался базисами чаще других:

$$a(x) = \text{mode}(b_1(x), b_2(x), \ldots, b_K(x)).$$

- Модификация для классификации — если в каждой точке $X = x$ нужно распределение вероятностей классов, можно для каждого класса посчитать среднюю предсказанную вероятность этого класса среди всех базисов:

$$P(y = c \mid x) = \frac{1}{K}\sum_{i=1}^{K} P_i(y = c \mid x).$$

Есть много модификаций того, как должна выглядеть агрегирующая функция. Но если говорить о классическом бэггинге, то база одна — среднее арифметическое для регрессии и мода для классификации.

Этап 2. Подбор и обучение базовых алгоритмов

Ансамбли со средним в качестве агрегатора дают лучшее качество в случае с более независимыми базисами. Можно было бы взять большое количество независимых обучающих выборок (200–300–500–1000 выборок), на каждой из них обучить отдельный базовый алгоритм и получить независимые оценки базисов (базисы, обученные на независимых выборках, выдают примерно независимые предсказания).

Однако на практике очень затратно получать сотни независимых обучающих выборок. Вместо этого используют бутстрэп.

INTERVIEW KIT

Бутстрэп (англ. bootstrap) — метод, при котором из одной исходной выборки формируется множество новых: каждая получается случайным выбором с возвращением. Бутстрэп-выборка содержит столько же строк, сколько исходная, — ровно N. Но часть объектов в неё попадает по нескольку раз, а часть не попадает вовсе.

Каждая из таких выборок сохраняет исходный размер, но может содержать повторяющиеся объекты. Это позволяет моделям обучаться на различных вариациях одного и того же набора данных (см. рисунок 18).

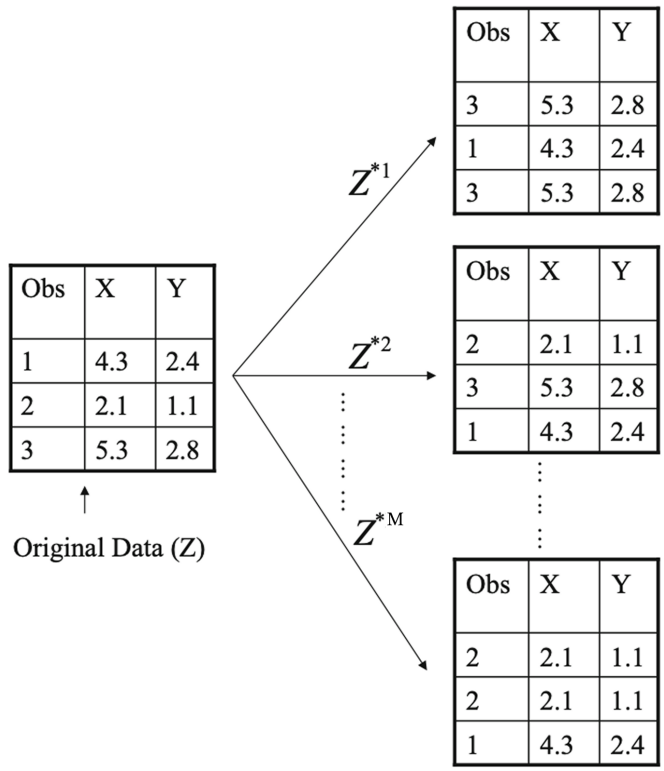


Рис. 18. Один из популярнейших методов ресемплирования — бутстрэп. Источник: ISLR, Hastie Tibshirani

INTERVIEW KIT

Бэггинг (англ. bagging, от Bootstrap AGGregatING) — это ансамблевый метод, при котором несколько моделей обучаются на различных бутстрэп-выборках исходных данных, а их предсказания агрегируются: усреднением для регрессии и модой (голосованием) для классификации (см. рисунок 19).

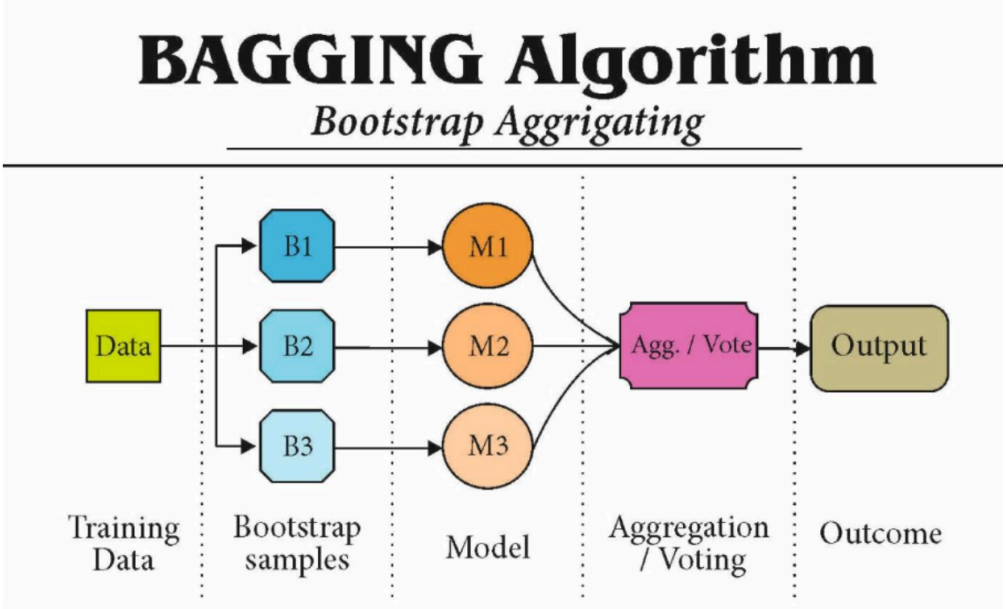


Рис. 19. Как работает бэггинговый ансамбль. Источник

Идея бэггинга

Каждый базовый алгоритм обучается на случайном подмножестве обучающей выборки, и эта случайность в виде бутстрэпа становится источником разнообразия базисов.

В классическом бэггинге базисы принадлежат одному классу алгоритмов (гомогенный ансамбль); например, базисы часто бывают деревьями решений.

Оценка качества алгоритма

Чтобы оценить качество итогового ансамбля, можно было бы использовать кросс-валидацию (CV). Однако алгоритм бэггинга даёт удобный способ получить внутреннюю оценку обобщающей ошибки без отдельной кросс-валидации. Способ строится на том, что вероятность попасть в бутстрэп-выборку для достаточно больших выборок равна примерно 0,632.

Это значит, что каждый базис обучается на N строках,

- среди которых примерно 63,2 % — уникальные объекты исходной выборки (остальные строки — повторы);
- оставшиеся $\approx 36{,}8$ % объектов в его обучении не участвуют вовсе. Именно они и называются out-of-bag для этого базиса.

Бутстрэп делает N независимых выборов с возвращением из N объектов. Вероятность, что конкретный объект не выбран за один раз, равна $(1 - 1/N)$. А что он не выбран ни разу за все N выборов — $(1 - 1/N)^N \to 1/e \approx 0{,}368$ при $N \to \infty$.

Значит, объект попадает в бутстрэп-выборку с вероятностью $\approx 1 - 1/e \approx 0{,}632$. Отсюда и берутся $\approx 63{,}2$ % уникальных «обучающих» объектов и $\approx 36{,}8$ % OOB-объектов у каждого базиса. Здесь речь именно о доле уникальных объектов — размер самой бутстрэп-выборки всегда равен N.

Эти $\approx 36{,}8$ % объектов на каждом базисе называются Out-of-Bag (OOB) наблюдениями. Рассмотреть это явление можно на рисунке 20.

Как происходит OOB-валидация бэггинга

Для каждого базиса есть свой набор OOB-объектов. Мы можем предсказать отклик для i-го наблюдения, используя только те базисы b_k, для которых объект x_i оказался OOB. Обычно таких моделей будет примерно $K/3$, где K — общее количество моделей.

Чтобы получить одно итоговое предсказание для i-го наблюдения, можно:

- если задача регрессии — усреднить все эти предсказания;
- если задача классификации — применить правило большинства.

Это даёт одно OOB-предсказание для данного наблюдения.

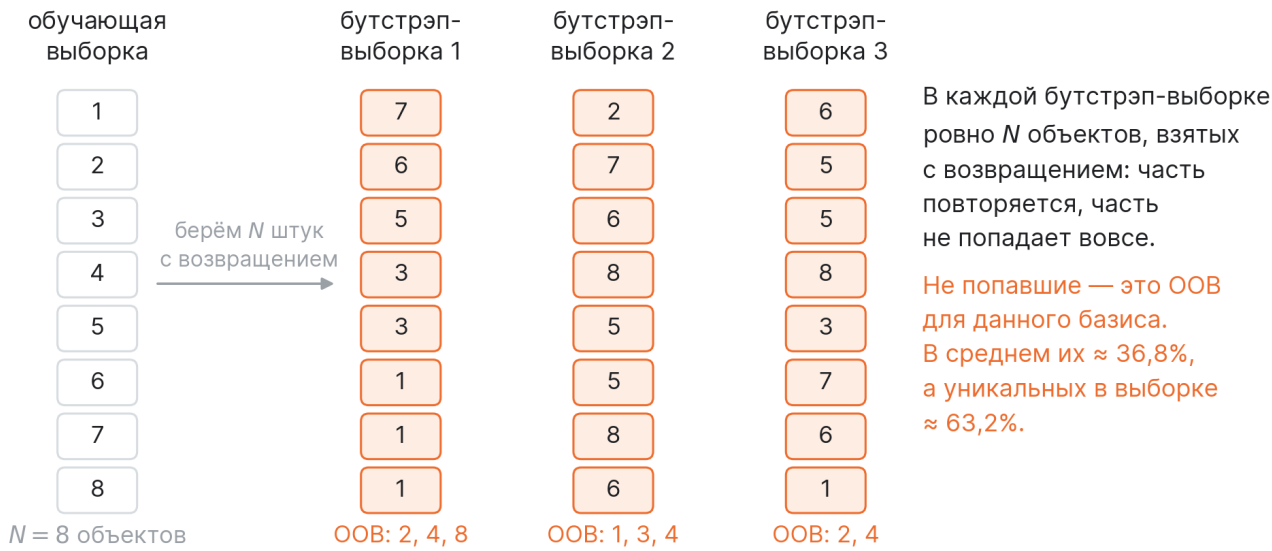


Рис. 20. В каждой бутстрэп-выборке N объектов, взятых с возвращением: часть повторяется, часть не попадает вовсе. Не попавшие — out-of-bag для этого базиса

Таким образом, для каждого из N наблюдений получаем OOB-предсказание:

$$a_{\text{OOB}}(x_i) = \frac{1}{|\{k : x_i \in \text{OOB}_k\}|} \sum_{k : x_i \in \text{OOB}_k} b_k(x_i).$$

Здесь:

- k — номер базиса,
- i — номер наблюдения.

Затем по этим предсказаниям считаем саму ошибку. Для регрессии — среднеквадратичную:

$$\text{MSE}_{\text{OOB}} = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - a_{\text{OOB}}(x_i)\right)^2.$$

Для классификации — долю неверных OOB-предсказаний. Полученная таким образом OOB-ошибка — это внутренняя оценка обобщающей ошибки ансамбля. Где отклик каждого наблюдения предсказывается только базисами, которые не видели объект при обучении, поэтому утечки нет. Но это именно оценка, а не тестовая ошибка в буквальном смысле: каждый объект оценивается ансамблем примерно из $K/3$ базисов вместо всех K.

Поэтому OOB обычно слегка пессимистична, а сами объекты — те же, что участвовали в обучении ансамбля. Полноценную независимую оценку по-прежнему даёт только отложенная тестовая выборка.

Формулы выше показывают, как OOB-оценка устроена. В `sklearn` её не нужно считать руками — достаточно попросить.

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

clf = BaggingClassifier(estimator=DecisionTreeClassifier(),
                        n_estimators=50,
                        bootstrap=True,    # OOB существует только при бутстрэпе
                        oob_score=True,
                        random_state=0)
clf.fit(X_train, y_train)

clf.oob_score_               # доля верных OOB-предсказаний — та самая величина из
 ↪ формулы
clf.oob_decision_function_   # сами OOB-предсказания, по одному на объект
```

Без `bootstrap=True` out-of-bag-объектов просто не существует, и `oob_score=True` вызовет ошибку.

Применение бэггинга

Разберём два случая: когда бэггинг хорошо работает и когда его лучше не использовать.

Случай 1. Пусть данные генерируются случайной функцией регрессии (истинная зависимость + шум):

$$y = x \sin x + \varepsilon, \qquad \text{где} \quad \varepsilon \sim \mathcal{N}(0, 9).$$

Посмотрим, как меняются предсказания решающих деревьев глубины 7 и ансамбля (бэггинга) на их основе в зависимости от обучающей выборки (посмотрим на дисперсию предсказаний).

- Сначала обучим 100 отдельных деревьев глубины 7, каждое — на своём случайном подмножестве данных (размером 20 объектов).
- Затем построим ансамбль: возьмём бэггинг из 10 таких деревьев и тоже 100 раз обучим его на случайных бутстрэп-выборках того же размера (20).

Визуализируем предсказания всех моделей на рисунке 21.

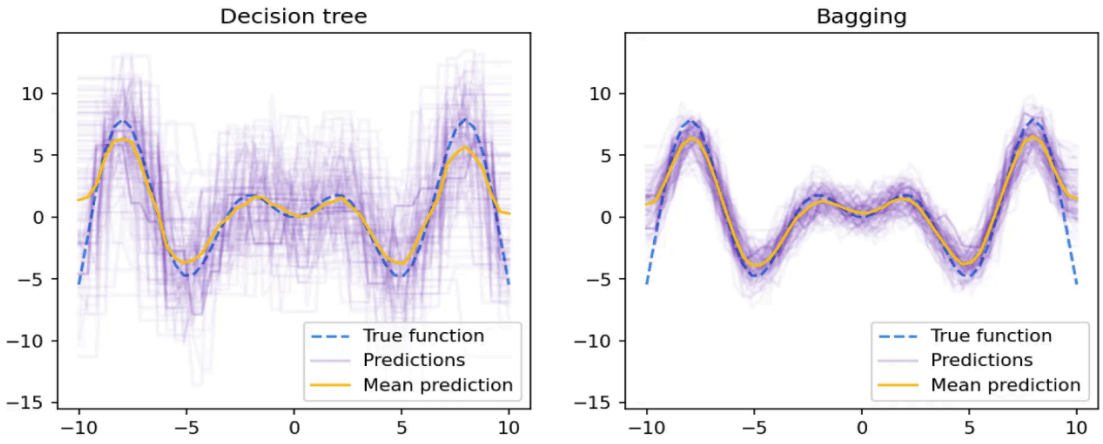


Рис. 21. Удачный пример построения бэггингового ансамбля на базовых деревьях решений. Источник

На рисунке 21 синяя функция — это истинная функция ($x \sin x$) регрессии. На левом графике предсказания отдельных деревьев почти не смещены (оранжевая функция средних предсказаний почти совпадает с синей истинной регрессией), но крайне волатильны — фиолетовые кривые отдельных деревьев сильно расходятся между собой. Это та самая высокая дисперсия, из-за которой одиночное дерево проигрывает на новых данных.

Теперь посмотрим на эффект от ансамблирования. Правый график показывает, что дисперсия предсказаний бэггинга в разы меньше, чем дисперсия отдельных деревьев. Смещение при этом не изменилось — у ансамбля оно такое же, как у одного дерева. Усреднение бьёт по разбросу и не трогает смещение — ровно то, что мы доказали формулами выше.

INTERVIEW KIT

Таким образом, этот пример показывает, что ансамблирование в виде бэггинга — это крайне эффективный инструмент в случаях, когда базовые модели ансамбля несмещённые, но крайне волатильные алгоритмы. Самый яркий пример базиса в бэггинге — это глубокое и переобученное дерево.

Случай 2. Пример задачи бинарной классификации. Обучающую выборку невозможно разделить линейно так, чтобы идеально отделить два класса. Тем не менее мы обучаем три логистических регрессии на разных подвыборках данных, а затем формируем бэггинг из этих трёх базисов.

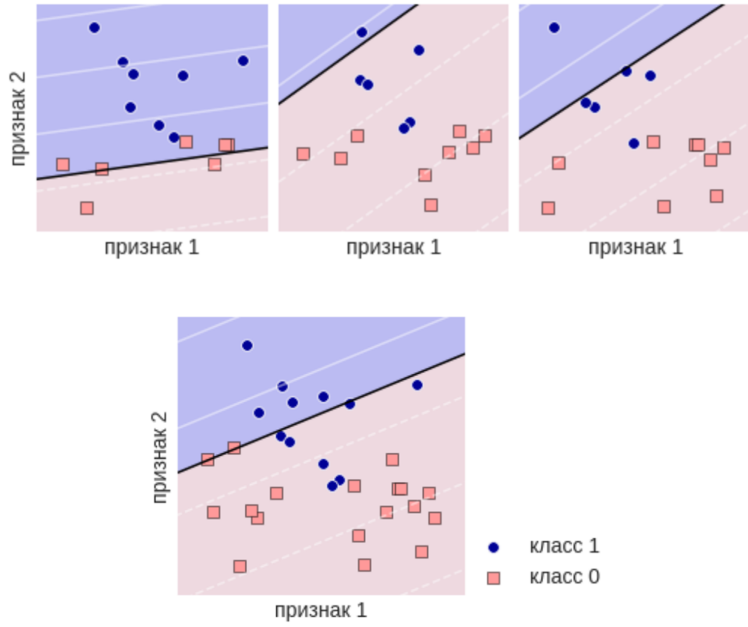


Рис. 22. Пример неудачного построения бэггингового ансамбля. Три базовых алгоритма (рисунки сверху) и результат бэггинга (рисунок внизу). Источник

INTERVIEW KIT

Логистическая регрессия ввиду своей линейности имеет низкую дисперсию, но зато часто страдает от высокого смещения.

Применение бэггинга на логистической регрессии (см. рисунок 22) не улучшает качество отдельной логистической регрессии.

Таким образом, в параллельных ансамблях — бэггинг, пастинг, случайный лес — базовые модели должны как можно лучше подстраиваться под обучающую выборку. В случае с последовательными ансамблями (например, бустинг) ситуация иная. Бустинги мы рассмотрим на следующих неделях.

Модификации бэггинга

Бутстрэп — это не единственный источник разнообразия. Случайной можно сделать не только выборку строк, но и выборку столбцов. Если каждый базис учить на всех объектах, но на случайном подмножестве признаков, деревья перестают строиться по одним и тем же признакам и корреляция между ними падает. Это метод случайных подпространств (англ. random subspaces). Именно на этой идее построен случайный лес, к которому мы придём чуть ниже.

ПРИМЕЧАНИЕ

Три родственника бэггинга

Каждый метод отличается от классического бэггинга объектом семплирования.

Пастинг (англ. pasting) — это подвыборка строк без возвращения. Внутри одной подвыборки объект встречается не больше одного раза (в бутстрэпе может повторяться). Чуть быстрее бэггинга, но базисы получаются более похожими друг на друга, поэтому дисперсия падает слабее. Оправдан, когда данных очень много.

Метод случайных подпространств (англ. random subspaces, или feature bagging) — это тот самый метод, который описан выше. Из p исходных признаков каждая модель видит $m < p$, а объекты берём все. Хорошо декоррелирует базисы,

> когда признаков много и они скоррелированы между собой. Но при слишком маленьком m базисы видят мало информации и смещение растёт, а OOB-оценка недоступна — базисы обучаются на всех наблюдениях. В случайном лесу эта же идея работает жёстче: там набор признаков разыгрывается заново в каждом узле, а не один раз на всё дерево.
>
> **Метод случайных участков** (англ. random patches) — это метод, при котором семплируют и строки, и столбцы сразу. Он даёт дополнительную декорреляцию и экономит память на «широких» и «длинных» данных, но настраивать приходится две доли — по строкам и по признакам.
>
> Все три собираются из того же конструктора `BaggingClassifier`, что и бэггинг, — как именно, показано в блоке кода ниже. Эти модификации понадобятся в домашнем задании: там нужно собрать каждую из них и разобраться, какие гиперпараметры за что отвечают.

Реализация в Python

Рассмотрим, как бэггинговые и родственные ансамбли реализуются в Python. В scikit-learn используем ансамблевый модуль `sklearn.ensemble` и класс этого модуля `BaggingClassifier` (возьмём классификацию, хотя в регрессионном `BaggingRegressor` все гиперпараметры те же).

```
from sklearn.ensemble import BaggingClassifier
from sklearn.svm import SVC

clf = BaggingClassifier(estimator=SVC(),  # Базовая модель (дефолт –
 ↪  DecisionTreeClassifier)
                        n_estimators=10,  # Число алгоритмов в ансамбле
                        max_samples=1.0,  # Размер подобучения (доля или число)
                        max_features=1.0, # Число/доля признаков для подобучения
                        bootstrap=True,   # Выбирать ли подобучение с возвращением
                        bootstrap_features=False, # Аналогичная опция для признаков
                        oob_score=False,  # Вычислять ли OOB-ошибку
                        warm_start=False, # Добавить базисы
                                          # к уже обученным
                        n_jobs=None)      # Параллельные вычисления

clf.fit(X, y)
```

С помощью такого конструктора можно инициализировать не только бэггинг (как мы и сделали в блоке кода), но и пастинг, Random Subspaces и Random Patches:

- для пастинга нужно указать `bootstrap=False`;
- для Random Subspaces меняем гиперпараметры `bootstrap=False` и `max_features= ...` на значение меньше единицы;
- для Random Patches с предыдущими модификациями ещё и изменить `max_samples= ...` тоже на значение меньше единицы.

```
pasting  = BaggingClassifier(bootstrap=False)
subspace = BaggingClassifier(bootstrap=False, max_features=0.5)
patches  = BaggingClassifier(bootstrap=False, max_features=0.5, max_samples=0.7)
```

ПРИМЕЧАНИЕ Бутстрэппить можно не только объекты, но и признаки.

3.2. Random Forest

Если в бэггинге на деревьях добавить ещё один источник случайности — на каждом разбиении узла выбирать лучший признак не из всех p признаков, а из случайного подмножества $m < p$, — получится случайный лес (random forest, RF).

ОПРЕДЕЛЕНИЕ

Случайный лес — это бэггинг над деревьями, в котором при каждом сплите кандидаты-признаки выбираются случайно. Коротко: RF = Bagging + случайные подпространства на уровне каждого сплита.

Зачем нужна эта дополнительная случайность

Признаковая рандомизация сильнее декоррелирует деревья. Если в данных есть один-два очень сильных признака, обычный бэггинг ставит их в корень почти каждого дерева — деревья становятся похожими, корреляция ρ растёт, и эффект ансамбля слабеет. Случайный лес «даёт слово» и более слабым признакам: разнообразие деревьев увеличивается, а дисперсия ансамбля падает сильнее (см. рисунок 23).

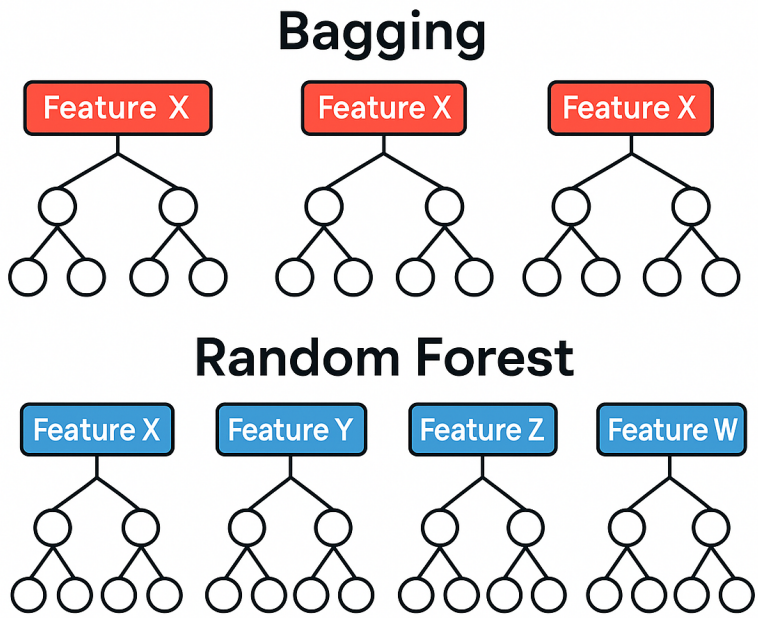


Рис. 23. Случай с одним сильным признаком: бэггинг всегда тянет его в вершины деревьев и повышает их корреляцию; случайный лес устраняет эту «монополию признака»

Этого пока достаточно, чтобы понять идею. Полный разбор случайного леса — алгоритм обучения, гиперпараметры (`n_estimators`, `max_features` и другие), важность признаков (MDI/MDA), ускоренная версия Extra Trees и практические рекомендации — мы вынесем в неделю 8, целиком посвящённую случайному лесу и бустингу.

4. СТЕКИНГ И БЛЕНДИНГ

Напомним, что ансамбль — это функция вида

$$a(x) = F(b_1(x), b_2(x), \dots, b_K(x)),$$

где F — это метаалгоритм, который мы до сих пор считали за базовое среднее арифметическое (регрессия) или моду (классификация).

Более того, до этого момента мы рассматривали $b_1(x), b_2(x), \dots, b_K(x)$ как базовые модели, каждая из которых по отдельности неустойчива (низкое смещение, но высокая дисперсия), однако при усреднении они выдают высокоточный результат.

Напрашиваются вопросы: почему F — это всего лишь среднее или мода? И почему базовые алгоритмы обязаны быть простыми и однотипными? Можно ли создать ансамбль, который не будет следовать этим правилам? Ответ — конечно, можно. Однако такие ограничения необязательны: можно создавать ансамбли, в которых метаалгоритм реализован сложной моделью, а базовые модели — произвольной сложности.

INTERVIEW KIT

Стекинг (англ. stacking, stacked generalisation) — это метод ансамблирования, при котором предсказания нескольких разнородных моделей (любой сложности) служат входными признаками для метамодели, обучающейся на этих предсказаниях.

ВАЖНО

Главное правило стекинга: метамодель обучается только на таких предсказаниях, которые базис сделал для объектов, не участвовавших в его обучении.

Всё остальное в стекинге — детали реализации, поэтому это правило нарушать нельзя. Если его нарушить и обучить метамодель на предсказаниях базисов по их же обучающим данным, то переобученный базис (например, неограниченное дерево) покажет там почти идеальное качество. А метамодель поверит и отдаст ему почти весь вес — а на новых данных этот базис окажется обычным, и ансамбль выродится в одну переобученную модель.

Именно ради выполнения этого правила и придуманы обе схемы:

- в стекинге — кросс-валидация с out-of-fold-предсказаниями;
- в блендинге — отложенная выборка.

Сравним механизмы стекинга с механизмами случайного леса.

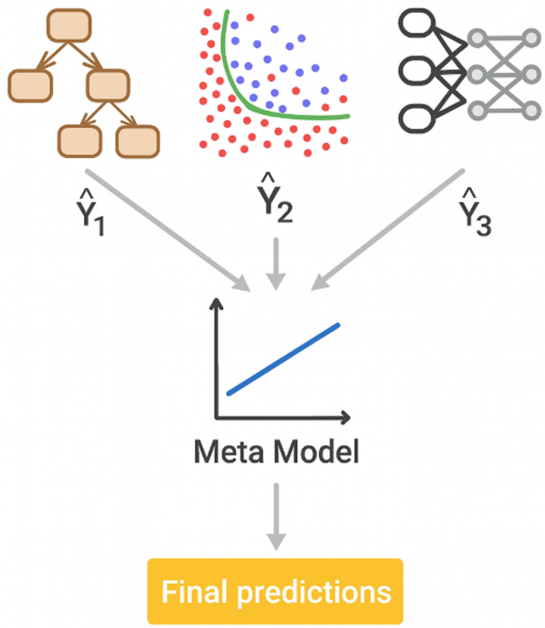


Рис. 24. Схема работы стекинга. Изображение создано в сервисе Claude Code

В сравнении со случайным лесом стекинг — это ансамбль более высокого уровня сложности. Задача стекинга — не просто усреднить ответы однотипных базисов, а обучиться на предсказаниях сильных и разнородных базовых алгоритмов, используя полноценную метамодель. Она определяет, насколько каждый из базисов «прав» в своих предсказаниях (см. рисунок 24).

	Случайный лес	Стекинг
Базовые алгоритмы	Сильные по отдельности (низкое смещение), но неустойчивые — обычно глубокие переобученные деревья	Обычно сильные, самодостаточные
Разнообразие базисов	Один и тот же класс моделей	Желательно разные классы моделей
Метаалгоритм	Базовое усреднение/голосование	Полноценная ML-модель
Источник разнообразия в ансамбле	Бутстрэп-выборки + случайные подмножества признаков	Разные классы моделей и типы гиперпараметров
Цель	Уменьшение дисперсии	Уменьшение дисперсии, а при удачном подборе базисов — и смещения
Скорость обучения и предсказания	Умеренная при небольшом количестве деревьев	Медленная: тренируются несколько «тяжёлых» моделей + метауровень; предсказание суммирует все базисы
Интерпретируемость	Низкая	Ещё ниже: множество разнородных моделей, метауровень усложняет объяснение
Риск переобучения	Низкий	Выше, особенно при сложных моделях или утёкших данных
Сложность реализации	Невысокая, хорошо работает без дополнительной настройки	Высокая — нужен подбор разнородных моделей, аккуратная Out-of-Fold-валидация, обучение метауровня и контроль утечки таргета
Точность	Высокая по умолчанию. Хороший baseline, но имеет «потолок» за счёт похожих базисов	Часто выше, чем у отдельных моделей: комбинирование сильных и разнородных базисов помогает выжать последние доли процента. Требует аккуратности — при похожих базисах или утечке выигрыша может не быть. Часто используется в Kaggle-соревнованиях

INTERVIEW KIT

Таким образом, метамодель учится взвешенно объединять предсказания разных базисов, одновременно сглаживая их случайные флуктуации и компенсируя систематические ошибки. За счёт этого:

- дисперсия обычно снижается благодаря тому, что метамодель усредняет (в прямом или «взвешенном» виде) разнородные прогнозы и тем самым убирает случайный шум каждой отдельной модели;
- смещение тоже может уменьшиться. И это не противоречит выводу из раздела про бэггинг. Напомним: там мы показали, что усреднение не меняет смещение, но опирались на два условия — базисы одинаково распределены (значит, смещение у всех одно и то же) и веса фиксированы (равны $1/K$).

В стекинге же не выполнено ни одно. Базисы разнородные, их смещения различаются по величине и знаку: линейная модель на каком-то участке систематически занижает, дерево там же завышает. А веса не фиксированы — метамодель подбирает их по данным. Поэтому она может найти комбинацию, в которой систематическая ошибка одного базиса гасится систематической ошибкой другого. При простом усреднении одинаково распределённых моделей это невозможно в принципе.

В результате стекинг обычно обобщает лучше, чем однотипный набор деревьев, но требует более сложного пайплайна обучения. Слово «обычно» здесь существенное: выигрыш появляется, когда базисы действительно разнородны и ошибаются по-разному, а метамодель обучена без утечки.

4.1. Как работает стекинг

Работа с базовыми моделями

В стекинге нет бутстрэпа,

- поэтому базисы не учатся на случайных подвыборках с возвращением. А значит, нет и OOB-наблюдений — взять OOB-предсказания для обучения метамодели как в бэггинге не получится;
- брать предсказания базиса на тех же данных, на которых он обучался, тоже нельзя — лучший результат покажет самый переученный базис, метамодель отдаст ему почти весь вес, и получится утечка таргета (data leakage).

Вместо этого используем кросс-валидацию. Важный момент: чтобы собрать метапризнак, базис обучается не на полном тренировочном датасете. Базис обучается k раз — каждый раз на $k-1$ фолдах. И предсказывает тот фолд, который не видел. На полном train, базис обучится заново, но это отдельный шаг, и к метапризнакам для train он отношения не имеет.



Рис. 25. Сквозная схема OOF-стекинга: базис учится на $k-1$ фолдах и предсказывает отложенный, из отложенных предсказаний собирается матрица метапризнаков, на ней учится метамодель — и только после этого базисы переобучаются на полном train, уже для теста

Посмотрим, как работает алгоритм стекинга на примере.

1. Возьмём небольшие тренировочную и тестовую выборку, где первую разобьём на 4 фолда для проведения кросс-валидации (шаг 1 на схеме выше).
2. Далее первый базовый алгоритм обучим на трёх фолдах, а таргеты четвёртого фолда предскажем обученной моделью (шаг 2 на схеме).
3. Так сделаем для каждого из четырёх фолдов и получим предсказания первой базовой модели для всех объектов (шаги 2–3 на схеме). Эти предсказания также часто называют Out-of-Fold-оценками (OOF-оценками). В итоге они формируют новый метапризнак z_1.

ПРИМЕЧАНИЕ

Пример OOF. Пусть в train выборке 8 объектов, разбитых на 4 фолда по 2. Первый базис b_1 учится на трёх фолдах и предсказывает отложенный, и так 4 раза. Каждое предсказание сделано моделью, которая не видела этот объект.

Объект	Фолд	b_1 обучен на	OOF-предсказание (z_1)
1, 2	F1	F2 + F3 + F4	0,31 / 0,44
3, 4	F2	F1 + F3 + F4	0,62 / 0,58
5, 6	F3	F1 + F2 + F4	0,27 / 0,71
7, 8	F4	F1 + F2 + F3	0,49 / 0,53

Эти 8 значений — один метапризнак z_1. Повторяем для $b_2, b_3, \ldots$ и получаем метатренировочный датасет. Так как каждое OOF-предсказание сделано моделью, которая не видела объект, то утечки таргета нет. Поэтому и нельзя брать предсказания базисов на тех же данных, на которых они обучались.

4. Повторяем шаги 2–3 для каждой базовой модели (шаг 3 на схеме).
   Столбцы $z_1, z_2, \ldots$ и есть тот самый метатренировочный (OOF) датасет: строк в нём столько же, сколько объектов в train. А столбцов — сколько базисов. Обрати внимание: метапризнаки для train собраны только по OOF-схеме.
5. Обучаем метамодель на метатренировочном (OOF) датасете (шаг 4 на схеме). Она видит только «честные» предсказания — те, что базис сделал для объектов, которых не было в его обучающей части. Главное правило стекинга соблюдено, утечки нет.
6. Отдельным шагом обучаем каждый базис заново — уже на всей тренировочной выборке (шаг 5 на схеме). Именно эти «полные» версии дают метапризнаки для тестовых данных и для любых новых объектов. Их предсказания и образуют метатестовый датасет — по одному вектору от каждого базиса. Метамодель заново не обучаем: она уже настроена на OOF-метапризнаках и остаётся как есть.

Итого получились два метадатасета:

- метатренировочный — собран на шагах 2–4 по кросс-валидации. На нём обучилась метамодель;
- метатестовый — собран на шаге 5 «полными» базисами. Его метамодель превращает в финальный ответ на тесте.

Принципиально здесь не то, когда обучены «полные» базисы. А то, что метапризнаки для train берутся из OOF, а не от них. Иначе метамодель получила бы предсказания для объектов, которые базисы уже видели, и весь смысл схемы пропал бы.

Всю эту схему sklearn умеет собирать сам.

```
from sklearn.ensemble import StackingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold

stack = StackingClassifier(
    estimators=\[('lr', LogisticRegression(max_iter=1000)),
                ('rf', RandomForestClassifier(n_estimators=100, random_state=0))\],
    final_estimator=LogisticRegression(),                 # метамодель
    cv=StratifiedKFold(5, shuffle=True, random_state=0),  # шаги 2-4 схемы
    stack_method='predict_proba',
    n_jobs=-1)

stack.fit(X_train, y_train)   # шаг 5 — переобучение базисов на полном train — класс
 ↪  делает сам
```

Полезно один раз собрать то же самое руками — тогда видно, где именно живёт каждый шаг схемы.

```
import numpy as np
from sklearn.model_selection import cross_val_predict

base = \[LogisticRegression(max_iter=1000),
        RandomForestClassifier(n_estimators=100, random_state=0)\]
cv = StratifiedKFold(5, shuffle=True, random_state=0)

# метапризнаки для train — только OOF (это и есть таблица с 8 объектами выше)
Z_train = np.column_stack(\[
    cross_val_predict(b, X_train, y_train, cv=cv, method='predict_proba')\[:, 1\]
    for b in base\])

# базисы переобучаются на полном train — уже для теста
for b in base:
    b.fit(X_train, y_train)
Z_test = np.column_stack(\[b.predict_proba(X_test)\[:, 1\] for b in base\])

meta = LogisticRegression().fit(Z_train, y_train)
meta.predict_proba(Z_test)\[:, 1\]
```

Метамодель

От каждой базовой модели есть вектор предсказаний (метапризнак). Цель метамодели — понять, в каких условиях (при каких наблюдениях) каждый из базисов предсказывает близкие к истине оценки, чтобы комбинировать сильные стороны каждой базовой модели.

Пример:

- простое усреднение — это когда «все эксперты голосуют поровну»;
- метамодель же — опытный капитан команды экспертов: он замечает, что один базис силён на одних объектах, другой — на других. И учится, кому доверять в какой ситуации, а не слушает всех одинаково.

Метамоделью может быть почти любой алгоритм, но по умолчанию берут самый простой — линейную модель с регуляризацией. Она подбирает каждому базису свой вес — это самый простой ответ на вопрос «кому доверять». Как учесть, что базис силён не на всех объектах, — во врезке ниже.

Пусть есть K базовых моделей, их OOF-предсказания собраны в матрицу $Z \in \mathbb{R}^{N \times K}$. Метамодель решает, например, задачу Ridge-регрессии:

$$\min_{w,b} \; L\big(y,\, Zw + b\big) \; + \; \lambda \, \|w\|_2^2.$$

Здесь:

- L для регрессии — MSE;
- L для бинарной классификации — логистическая лог-потеря;
- w — веса моделей.

Если добавить ограничения $w_k \geq 0, \quad \sum_{k=1}^{K} w_k = 1$, то задача превращается в обучаемое взвешенное усреднение:

$$a(x) = w_1 \cdot b_1(x) + \cdots + w_K \cdot b_K(x).$$

Стандартно с линейными моделями часто идёт регуляризация (например, в sklearn в задачах стекинга регрессии по умолчанию берётся Ridge-регрессия), чтобы избежать переобучения и смягчить мультиколлинеарность, которая присуща всем метапризнакам (так как они являются результатом работы обученных базовых моделей).

ПРИМЕЧАНИЕ

Дополнительно: когда метамодель сложнее линейной. Линейная метамодель даёт каждому базису один общий вес. Иногда этого мало: базис бывает силён на одной части объектов и слаб на другой. Тогда берут метамодель, которая меняет веса от области к области, — дерево или градиентный бустинг. И реже нейросеть, которая подбирает свою смесь базисов для каждого объекта.

Совсем редко в роли метамодели встречаются SVM, kNN и другие ядровые методы — обычно в случаях, когда базисов мало, а граница между классами сложная. Плата всегда одна: чем сложнее метамодель, тем выше риск, что она переобучится на метапризнаках, которых всего K штук.

В итоге обученную метамодель можно использовать для предсказания таргетов на тестовых данных. В общем виде процесс предсказания нового объекта следующий.

1. Получаем предсказание таргета объекта от каждого из базисов, обученных на полной обучающей выборке. Итог — вектор из K значений метапризнаков.
2. Модель — метаалгоритм, вводные данные — вектор метапризнаков, выходные данные — значения таргета для этого наблюдения.

Получаем готовый стекинговый ансамбль. На рисунке ниже изображён пример работы стекинга:

- цветные области показывают, к какому классу (синему или красному) отнесёт точку тот или иной метод;
- белые и полупрозрачные линии — уровни уверенности (контуры вероятности).

Так можно визуально сравнить простые базовые модели и то, как их объединение через Blending и Stacking даёт более «гладкие» и надёжные решения.

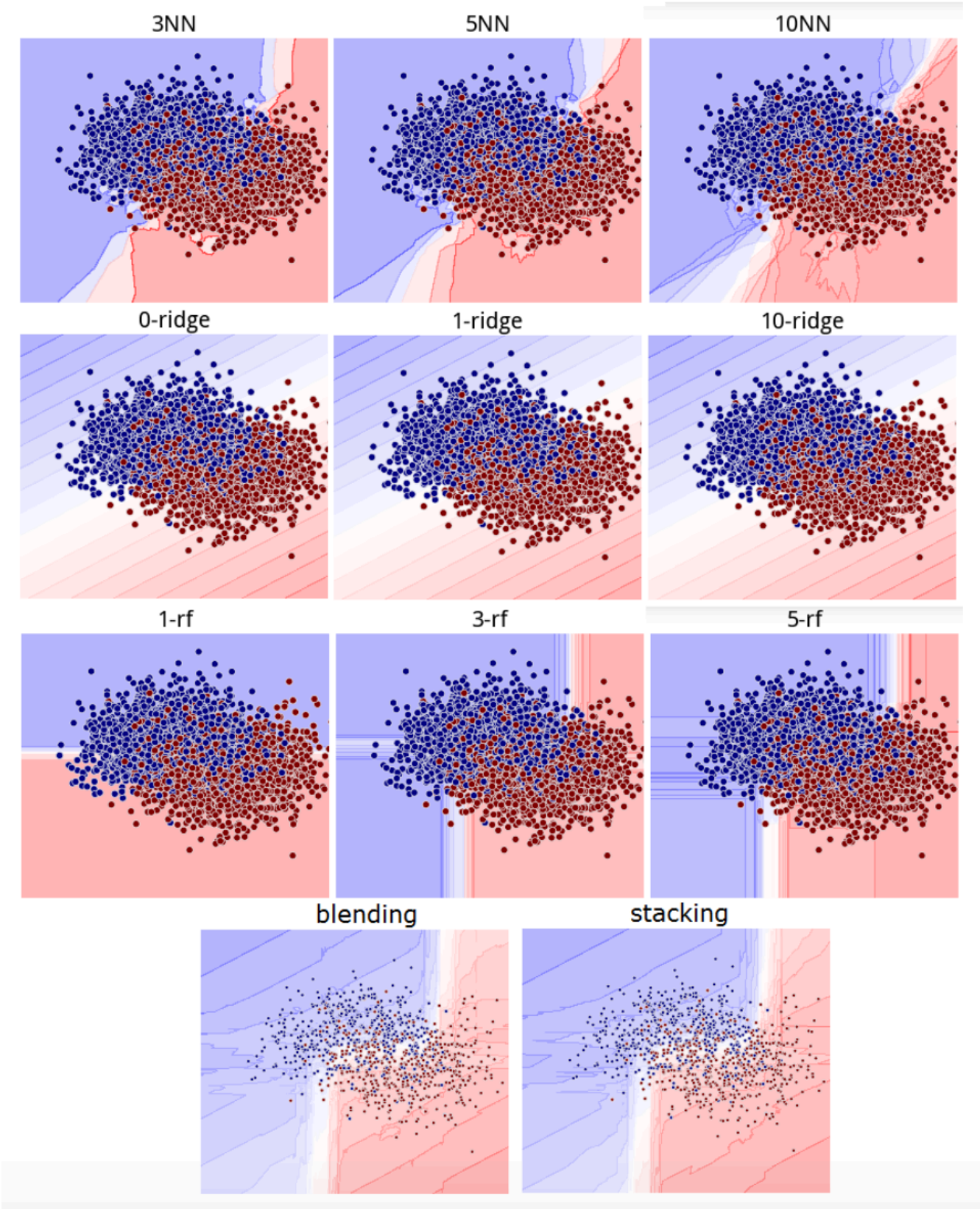


Рис. 26. Пример работы ансамблевых стекинга и блендинга (разберём далее) в случае бинарной классификации. Источник

Особенности стекингового ансамбля

1. Выбор и разнообразие базовых моделей. Стекинг особенно ценен, когда базисы разной природы — то есть деревья, линейные модели и градиентные бустинги обучены на разных преобразованиях признаков. Разнообразие снижает корреляцию ошибок и расширяет информацию, доступную метауровню.
2. Ограничения по качеству. Стекинг не всегда существенно повышает качество лучшего из базовых алгоритмов (см. рисунок 27).

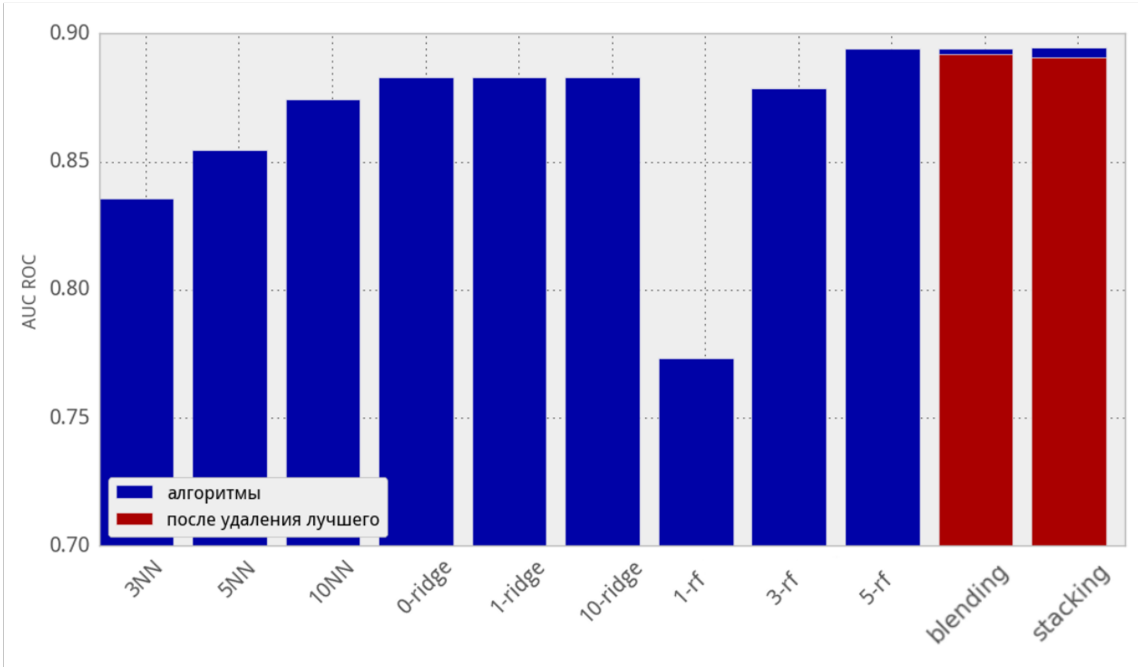


Рис. 27. Синие столбцы — базисы, красные — ансамбли после удаления лучшего алгоритма. Стекинг и блендинг не всегда улучшают качество базовых алгоритмов. Источник

3. Метапризнаки коррелируют между собой. OOF-предсказания базисов могут быть сильно коррелированы между собой, поэтому метапризнаки оказываются избыточными, а веса метамодели — нестабильными. Риск переобучения особенно растёт при сложной метамодели, поэтому обычно используют простую модель с регуляризацией.
4. Что настраивают. Число фолдов k: чем больше, тем чище OOF-предсказания, но тем дольше обучается пайплайн. И силу регуляризации метамодели — L2-штраф λ в Ridge или параметр C в логистической регрессии.
5. Требования к размеру выборки. Любой двухуровневый ансамбль плохо чувствует себя на совсем маленьких данных: метапризнаков мало, метамодель легко переобучается, а шум OOF-оценок может перекрыть выгоду от ансамблирования. Чем больше выборка, тем стабильнее OOF-оценки. Но если данных немного, а двухуровневый ансамбль всё-таки нужен, OOF-схема предпочтительнее блендинга: она не отрезает часть выборки в holdout и даёт метапризнак для каждого объекта. На малых данных особенно важно держать метамодель простой и регуляризованной и тщательно подбирать базисы.

ПРИМЕЧАНИЕ

Дополнительно: что ещё настраивают в стекинге

- Стабилизация — делают несколько независимых разбиений на фолды и усредняют результат или добавляют к метапризнакам небольшой шум.
- Несколько метауровней — выходы одного стекинга подают на вход следующему. Такой приём популярен в соревновательном машинном обучении; в продукте такую конструкцию дорого поддерживать и почти невозможно объяснить.
- Исходные признаки вместе с метапризнаками — иногда помогает, но чаще ведёт к переобучению, поэтому в проде так делают осторожно и со строгой валидацией.

4.2. Как работает блендинг

ОПРЕДЕЛЕНИЕ

Блендинг (англ. blending) — это двухуровневый ансамбль, в котором обучающая выборка один раз делится на две части: базовые модели учатся на первой, а метамодель — на их предсказаниях для второй, отложенной (holdout).

В отличие от стекинга здесь нет k-fold-генерации OOF-прогнозов: достаточно одного деления данных. Главное правило при этом соблюдается — метамодель видит предсказания только для тех объектов, которых базисы не видели при обучении. Плата за простоту в том, что базисы учатся не на всей выборке, а метамодель получает лишь столько объектов, сколько попало в holdout. Чем меньше данных, тем болезненнее эта плата: отрезать 10–20% объектов от небольшого датасета дорого и для базисов, и для метамодели, которой остаётся совсем мало строк.

ПРИМЕЧАНИЕ

Блендинг — это простейшая форма стекинга. Метамоделью обычно служит простая линейная модель, обученная на holdout-предсказаниях базисов.

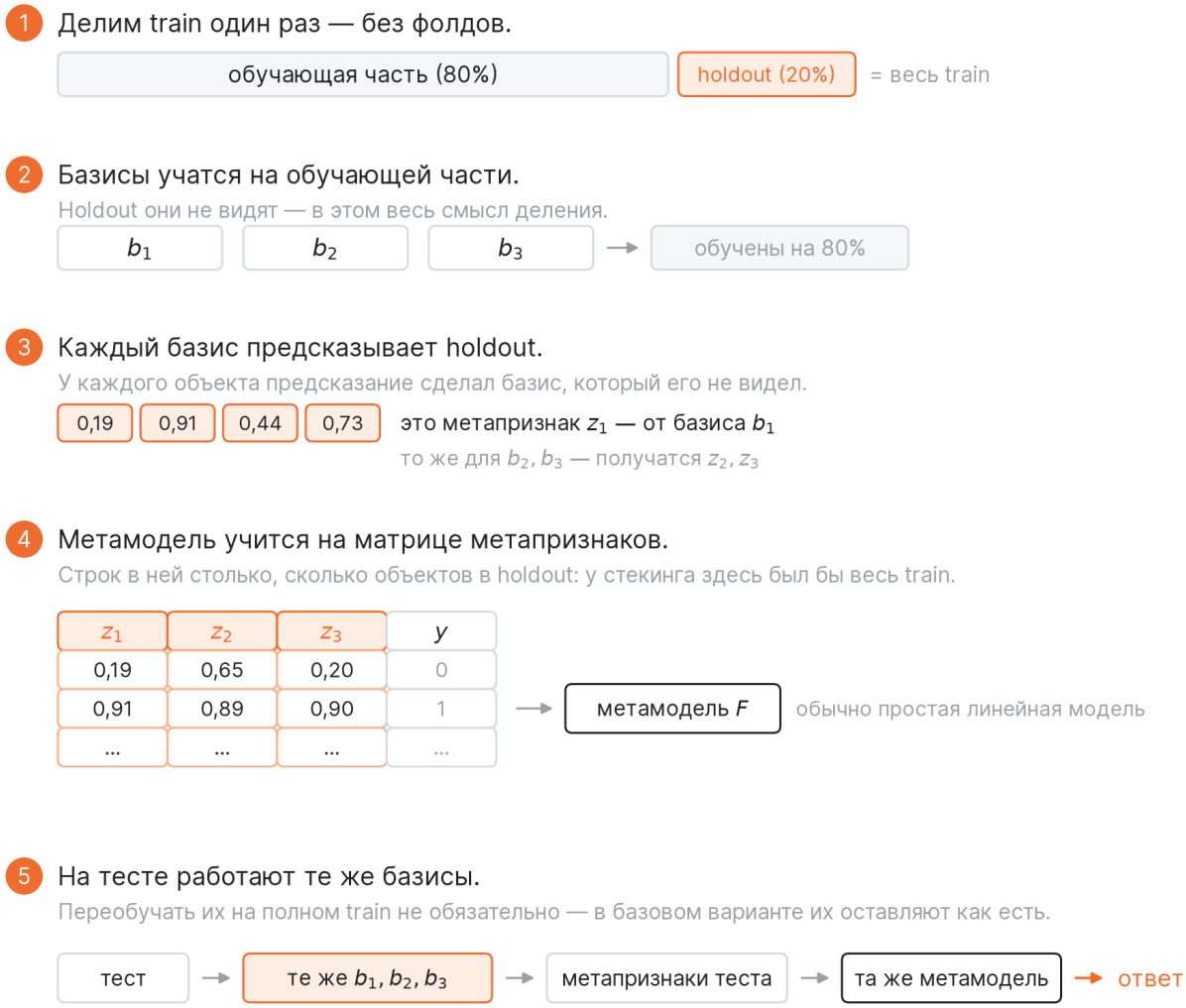


Рис. 28. Механизм блендинга: train делится один раз, базисы учатся на первой части и предсказывают holdout — из этих предсказаний собирается матрица метапризнаков z_1, z_2, z_3, на ней учится метамодель, а на тесте работают те же базисы

INTERVIEW KIT

Блендинг работает в те же пять шагов, что на схеме.

1. Обучающая выборка (train) один раз делится на обучающую часть и holdout — без фолдов.
2. Базовые модели обучаются на обучающей части; holdout они при этом не видят.
3. Каждый базис предсказывает holdout — эти предсказания и есть метапризнаки $z_1, z_2, \dots, z_K$.
4. Из них собирается метатренировочная выборка, и на ней обучается метамодель.
5. На тесте работают те же базисы и та же метамодель: базисы дают метапризнаки теста, метамодель превращает их в ответ.

ПРИМЕЧАНИЕ

В базовом варианте блендинга базисы, обученные на первой части выборки, остаются окончательными: именно они потом дают метапризнаки для тестовых данных. В стекинге иначе — там базисы после сбора OOF-предсказаний обучаются заново на всей обучающей выборке, и метапризнаки для теста дают уже эти «полные» версии.

Из-за этого в стекинге есть встроенное расхождение: метамодель подбирала веса под базисы, обученные на $k-1$ фолдах. А применяются они к базисам, обученным на всём train, — чуть более сильным. На практике с этим мирятся: предсказания «полной» модели обычно близки к OOF-предсказаниям, а обучение на всех данных чаще всего немного повышает качество.

В блендинге же повторное обучение базисов на полной выборке — это вопрос выбора. Оставишь исходные — расхождения нет, обучишь заново — базисы сильнее, но появляется расхождение. На практике встречаются оба варианта. Главное — это понимать, какой из них используется, и не сравнивать блендинг со стекингом, забыв про эту разницу.

Готового класса для блендинга в `sklearn` нет. И на контрасте видна причина сложности стекинга: здесь всё умещается в несколько строк.

```
import numpy as np
from sklearn.model_selection import train_test_split

X_base, X_hold, y_base, y_hold = train_test_split(
    X_train, y_train, test_size=0.2, random_state=0, stratify=y_train)

base = \[LogisticRegression(max_iter=1000).fit(X_base, y_base),
        RandomForestClassifier(n_estimators=100, random_state=0).fit(X_base, y_base)\]

# метапризнаки — предсказания для holdout, которого базисы не видели
Z_hold = np.column_stack(\[b.predict_proba(X_hold)\[:, 1\] for b in base\])
blender = LogisticRegression().fit(Z_hold, y_hold)

# на тесте работают те же базисы, переобучать не обязательно
Z_test = np.column_stack(\[b.predict_proba(X_test)\[:, 1\] for b in base\])
blender.predict_proba(Z_test)\[:, 1\]
```

4.3. Стекинг против блендинга

Рассмотрим, чем друг от друга отличаются стекинг и блендинг и в каких ситуациях каждый из этих ансамблей используется.

	Stacking	Blending
Потенциал качества	Часто лучший: метамодель учится взвешивать ошибки базовых моделей	Хороший, но ограничен
Риск переобучения	Средний: зависит от сложности метауровня и числа OOF-фич	Средний: ниже при большом holdout (≥20%), но может вырасти при маленьком
Утечка данных	Минимальная при правильном k-fold OOF	При корректном разделении утечки нет; основной риск — нерепрезентативный holdout и высокая зависимость результата от одного разбиения
Потеря обучающих данных	Нет, используются все примеры через k-fold	10–20% уходят в holdout, модели учатся на меньшем объёме
Вычислительные затраты	Высокие: $k\times$ количество моделей + метауровень	Низкие: один прогон + простая агрегация
Скорость итераций	Медленная: длинный цикл обучения	Быстрая: можно «крутить» веса почти онлайн
Сложность имплементации	Сложно: много пайплайнов, хранение OOF-фич	Просто: один раз делим датасет и обучаем метамодель на holdout-предсказаниях
Чувствительность к удачной выборке	Низкая: k-fold сглаживает разброс	Высокая: один неудачный holdout просадка
Когда подходит	Выборкой жертвовать нельзя и нужна максимальная метрика; есть время на k-кратное обучение базисов	Данных достаточно, чтобы отдать 10–20% в holdout; нужен быстрый прототип или частое обновление модели в проде

5. ЕСЛИ КРАТКО

- Цель ML-алгоритма — не обнулить ошибку на обучении, а минимизировать тестовую ошибку. Здесь нулевая train-ошибка часто означает слабое обобщение.
- Переобучение — это когда слишком гибкая модель ловит шум обучающей выборки. Недообучение — слишком жёсткая модель не улавливает структуру данных. Оба случая повышают тестовую ошибку.
- Тестовая ошибка раскладывается на три части: неустранимый шум + смещение^2 + дисперсия. Смещение — систематическое отклонение, дисперсия — чувствительность к конкретной выборке.
- Изменение сложности модели задаёт компромисс bias–variance: обычно с ростом сложности смещение падает, а дисперсия растёт; тестовая ошибка имеет U-образную форму (но бывают исключения — например, double descent).
- Деревья особенно склонны к переобучению (высокая дисперсия); её сдерживают контролем сложности (pre-/post-pruning). Но одиночное дерево всё равно даёт средний результат — это мотивирует ансамбли.
- Ансамбль $a(x) = F\big(b_1(x), \dots, b_K(x)\big)$ объединяет много базовых моделей, чтобы повысить точность и устойчивость.
- В параллельных ансамблях усреднение не меняет смещение, но снижает дисперсию тем сильнее, чем ниже корреляция ρ между базисами. Отсюда главный принцип: базисы должны ошибаться на разных объектах.
- Bagging = обучение базисов на бутстрэп-выборках + усреднение (регрессия) или голосование (классификация). Работает, когда базисы почти несмещённые, но «прыгучие» (глубокие деревья).
- OOB — внутренняя оценка обобщающей ошибки, которая достаётся бесплатно: каждый объект оценивается только теми базисами, что не видели его при обучении. Отложенный тест она не заменяет.
- Random Forest = Bagging + случайный выбор признаков на каждом сплите; сильнее декоррелирует деревья. Подробно — в неделе 8.
- Stacking — предсказания нескольких (обычно сильных и разнородных) базисов становятся метапризнаками, на которых обучается метамодель. Чтобы избежать утечки, метапризнаки собирают через OOF (кросс-валидацию): базис обучается на $k-1$ фолдах и предсказывает отложенный. На всём train базисы обучают заново, и их предсказания идут только на тест и новые объекты. Blending — упрощённая версия: метамодель учится на одном отложенном holdout-множестве, но этот holdout выпадает из обучения базисов, что особенно чувствительно на малых данных.
- Главный плюс стекинга — умная агрегация: метамодель сама подбирает веса базисам и часто обыгрывает простое усреднение. Главные минусы всех ансамблей — рост сложности, потеря интерпретируемости и вычислительные затраты при обучении и инференсе.